
THE KV CACHE IS THE NEW MEMORY WALL

Tejinder Singh 

Dell Technologies, Santa Clara, CA 95054, USA

Singh.Tejinder@Dell.com

ABSTRACT

Autoregressive Large Language Model inference at long context is bounded by memory bandwidth rather than arithmetic throughput, and the binding resource shifts from model weights to the Key-Value (KV) cache as sequence length grows. For Llama-3-70B in BF16, the 140 GB weight footprint alone exceeds the 80 GB HBM of a single accelerator, and the KV cache of one 128k-token sequence adds 42 GB on top. Techniques that compress, evict, page, share, or offload KV state have proliferated, yet reported gains are measured under mutually inconsistent workloads, hardware targets, and quality metrics, which prevents rigorous cross-paper comparison. This Systematization of Knowledge paper unifies the field on analytical ground, with a measurement protocol that keeps derived and reported claims strictly separated. We formalize the exact memory traffic of decode-phase attention and derive closed-form expressions for arithmetic intensity as a decaying function of context length, parameterized by hardware topology for NVIDIA H100, NVIDIA B200, and AMD MI300X, including per-die bandwidth partitioning on multi-die packages and the crossover context lengths at which KV traffic overtakes weight traffic. We classify the literature into five domains: quantization, token eviction, KV paging, prefix caching, and heterogeneous memory tiering. We then evaluate a representative method from each domain on common workloads at 128k context under one measurement protocol, reporting compression factor, memory footprint, derived throughput-speedup bounds, and as-reported quality deltas, with the two evidence classes never mixed. The central finding is a three-regime structure: below a hardware-specific crossover context length, weight traffic dominates and KV compression yields negligible end-to-end speedup. Beyond the crossover, KV traffic dominates and each domain trades a measurable quality delta for bandwidth savings that approach the roofline bound. Paging and prefix sharing are lossless with respect to model output but address capacity rather than bandwidth. Quantization and eviction reduce bandwidth directly, with quality degradation that accelerates below 4-bit precision for naive schemes and that turns discontinuous for eviction on position-sensitive tasks, while structure-aware methods extend both floors. Heterogeneous tiering converts the bandwidth wall into an interconnect problem bounded by PCIe or NVLink fabric throughput rather than HBM. We conclude with design rules for selecting a compression domain given a hardware target, context length, and quality budget, and we enumerate open problems in the co-design of compression formats with attention kernels and serving schedulers.

Keywords KV cache · LLM inference · Memory wall · Roofline model · KV quantization · Token eviction · Paged attention · Prefix caching · Memory tiering

Contents

1	Introduction and the Decode-Phase Memory Wall	4
1.1	Decode-Phase Arithmetic Intensity	4
1.2	KV-Cache Growth with Context Length	4
1.3	Five Fragmented Mitigation Families	4

1.4	Comparability Deficiencies in the Published Evidence Base	5
1.5	Systematization Goal and Contributions	5
2	KV Cache Anatomy and a Five-Domain Taxonomy of Compression	6
2.1	Exact KV-Cache Structure of Autoregressive Transformers	6
2.2	A Unified Footprint Model	6
2.3	Domain I. Precision Reduction of Stored Keys and Values	7
2.4	Domain II. Token Eviction and Retention Policies	7
2.5	Domain III. Paged Block Management of KV Memory	8
2.6	Domain IV. Prefix Sharing Across Requests	8
2.7	Domain V. Heterogeneous Tiering of KV State	8
2.8	Orthogonality and Composition of the Five Domains	8
3	Analytical Derivations of Context-Dependent Arithmetic Intensity	8
3.1	Exact FLOP and Byte Accounting for a Decode Step	9
3.2	Arithmetic Intensity as a Function of Context Length and Batch	9
3.3	Hardware Topology and Per-Die Bandwidth Partitioning	10
3.4	Crossover Context Lengths	11
3.5	Worked Bounds and the Speedup Structure of Compression	12
3.6	Formal Statements of the Analytical Results	12
4	Comparative SoK Analysis Across the Five Compression Domains	15
4.1	Workload and Metric Definitions	15
4.2	The SoK Comparison Matrix	15
4.3	Precision Reduction Results	15
4.4	Token Eviction Results	16
4.5	Paging, Prefix Sharing, and Tiering Results	16
4.6	Cross-Domain Synthesis and the Speedup-Quality Frontier	16
5	Cross-Cutting Challenges and Open Research Directions	17
5.1	The Three-Regime Map Revisited	17
5.2	Compression-Aware Attention Kernels	17
5.3	Scheduler Co-Design, Disaggregation, and Placement	18
5.4	Evaluation Gaps in the Published Evidence Base	18
5.5	Open Research Directions	18
6	Conclusion and Design Rules	19
6.1	Summary of Findings	19
6.2	Design Rules for Domain Selection	19
6.3	Closing Remarks	20
A	Full Derivations	21

A.1	Per-Kernel FLOP and Byte Accounting for One Decode Step	21
A.2	Byte Accounting and Kernel Assumptions	22
A.3	Prefill Cost and the Worked Recompute Bound	22
A.4	Evaluation of the Topology Model	23
B	Per-Model and Per-Device Constants	24
B.1	Per-Token KV Bytes of Table 1	24
B.2	Parameter Counts Used in the Derivations	24
B.3	Crossover and Capacity Arithmetic	24
B.4	Ridge Points, Fixed Point, and Rate Ceilings	25
B.5	Sharing and Transfer Arithmetic of Section 4	25
C	Notation, Units, and Measurement Protocol	25
C.1	Notation	25
C.2	Units and Numerical Conventions	26
C.3	Evidence Classes and Mixing Rules	26
C.4	Workload Parameterization and Measurement Assumptions	26

1 Introduction and the Decode-Phase Memory Wall

Autoregressive Large Language Models generate text one token at a time. Every decode step executes a full forward pass over the model and attends over the Key-Value (KV) cache, the tensor pair that stores the key and value projections of every token processed so far [1]. The arithmetic structure of this step is fixed by the transformer architecture, and its cost structure is fixed by the memory hierarchy of the accelerator that executes it. Test-time reasoning workloads, which generate extended chains of intermediate tokens before producing a final answer, push per-request context lengths from 10^3 toward 10^5 to 10^6 tokens [2]. Under these conditions the cost of decode is the cost of moving bytes from High Bandwidth Memory (HBM) to the compute units, not the cost of arithmetic, a bound that recurs at every layer of the deployment stack [3]. This paper systematizes the techniques that attack that bottleneck.

1.1 Decode-Phase Arithmetic Intensity

The decode phase is memory bound by construction. At batch size one, a forward pass over a model with P parameters stored at width w_p bytes performs approximately $2P$ floating point operations and moves approximately $P \cdot w_p$ weight bytes, giving an arithmetic intensity of $2/w_p$ FLOP per byte, or 1 FLOP/B for BF16 weights. The ridge point of an NVIDIA H100 SXM, with 989.5 TFLOP/s of dense BF16 throughput and 3.35 TB/s of HBM3 bandwidth, sits at $I^* \approx 295$ FLOP/B [4, 5]. Decode at batch size one therefore operates at $1/295$ of the ridge, and no kernel-level improvement can raise sustained compute utilization above a fraction of one percent.

Attention over the KV cache sits in the same regime. One query token attending over s cached tokens costs $4sd_h$ FLOPs per head, excluding softmax terms of lower order in d_h , and reads $2sd_h w_{kv}$ bytes per head, an intensity of $2/w_{kv}$ FLOP/B, again 1 FLOP/B in BF16. Grouped-Query Attention raises this figure by the group factor $g = n_q/n_{kv}$ because one stored key-value pair serves g query heads, yet even at $g = 8$ the resulting 8 FLOP/B remains more than an order of magnitude below the H100 ridge [6].

Batching does not rescue the long-context regime. Weight reads are shared across the batch, so weight-side intensity grows linearly in batch size b and reaches the ridge only near $b \approx 295$ on H100 with BF16 weights. KV reads are not shared. Every sequence owns its cache, so KV bytes scale with b exactly as attention FLOPs do, and the attention-side intensity is invariant to batch size. As context length grows, KV traffic overtakes weight traffic, and the batch-size lever that serving systems rely on for throughput loses its effect. Section 3 derives this crossover exactly.

1.2 KV-Cache Growth with Context Length

The KV cache grows linearly in context length. For a model with L layers, n_{kv} key-value heads per layer, head dimension d_h , and element width w_{kv} bytes, the cache of one sequence of length s occupies

$$B_{kv}(s) = 2L n_{kv} d_h s w_{kv}, \quad (1)$$

where the factor of two accounts for keys and values. For Llama-3-70B ($L = 80$, $n_{kv} = 8$, $d_h = 128$) in BF16, Eq. (1) yields 327,680 bytes per token. The 140 GB BF16 weight footprint already exceeds the 80 GB HBM of a single accelerator, and one 128k-token sequence adds 42 GB of KV state on top [2, 5]. At 10^6 tokens the same sequence requires 328 GB, the aggregate HBM of more than four H100 devices. Architectural variants compress the constant but not the scaling law. Multi-Query Attention and Grouped-Query Attention reduce n_{kv} by sharing key-value heads across query heads [7, 6], and Multi-head Latent Attention projects keys and values into a low-rank latent representation [8]. All three leave B_{kv} linear in s . Figure 1 plots Eq. (1) for the Llama-3 family against single-device HBM capacities. The curves cross device capacity at context lengths that production reasoning workloads already exceed.

1.3 Five Fragmented Mitigation Families

Five families of techniques attack the KV-cache wall, each through a distinct mechanism and each evaluated largely in isolation. *Quantization* stores keys and values at reduced precision, from 8-bit down to 2-bit per element [9, 10]. *Token eviction* discards cached tokens predicted to carry little future attention weight, retaining sinks and heavy hitters [11, 12, 13]. *KV paging* manages the cache in fixed-size blocks to eliminate fragmentation and enable fine-grained sharing across sequences [14]. *Prefix caching* reuses the KV state of shared prompt prefixes across requests [15]. *Heterogeneous memory tiering* migrates KV state across HBM, host DRAM, and NVMe storage, substituting interconnect bandwidth for HBM residency [16, 17, 18]. The mechanisms are complementary in principle, yet the literature evaluates them under mutually inconsistent context lengths, batch sizes, hardware targets, and quality metrics, so reported

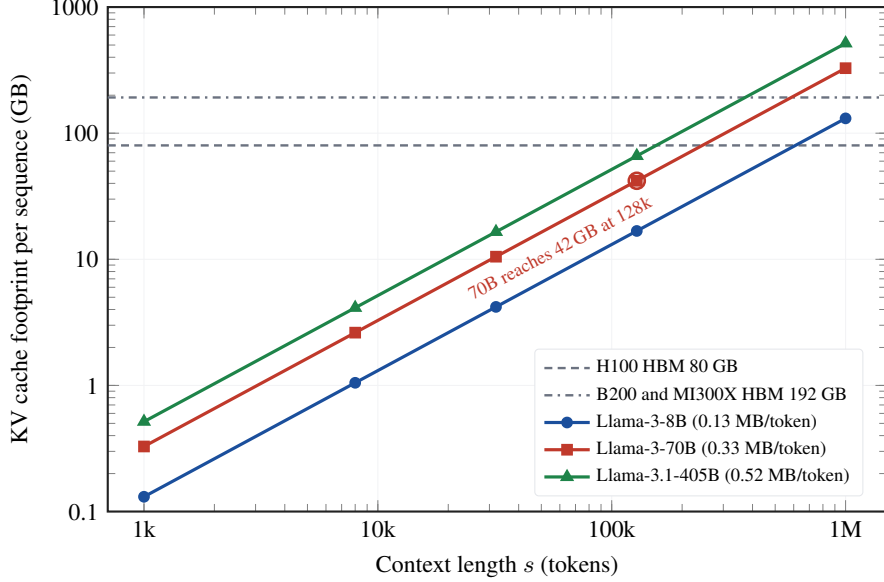


Figure 1: KV cache footprint per sequence versus context length for the Llama-3 family in BF16, computed from Eq. (1) [2]. Horizontal reference lines mark single-device HBM capacity. A Llama-3-70B sequence crosses one 80 GB H100 at 244k tokens of context and a Llama-3.1-405B sequence crosses it at 155k. At 10^6 tokens the 70B and 405B caches reach 328 GB and 516 GB, exceeding the aggregate HBM of four and six H100 devices respectively.

gains cannot be composed and can barely be compared. Xu et al. catalog this space descriptively [19]; our aim is not another catalog but the analytical frame that makes the catalog comparable.

1.4 Comparability Deficiencies in the Published Evidence Base

Three methodological deficiencies recur across the five families. First, there is no shared analytical model of KV memory traffic, so papers report speedups against baselines whose byte counts differ through batching policy, kernel choice, and cache layout. Second, hardware topology is ignored. NVIDIA B200 packages two compute dies behind an NV-HBI die-to-die link, and AMD MI300X partitions compute across eight accelerator chiplets, so aggregate HBM bandwidth is not uniformly accessible to every streaming multiprocessor [20, 21]. A technique that reduces KV bytes by a fixed factor can yield markedly different end-to-end speedups depending on how cache blocks map onto dies and chiplets. Third, quality is reported on incompatible scales. Perplexity on one corpus, needle retrieval accuracy on another, and downstream task scores on a third, with almost no study reporting quality and throughput under one protocol. Almost every published number is correct in isolation. Almost none are jointly interpretable.

1.5 Systematization Goal and Contributions

The goal of this paper is to place the KV-cache mitigation literature on analytical and empirical ground that permits rigorous comparison. We make four contributions.

1. We formalize the KV-cache structure of autoregressive transformers and classify the mitigation literature into five domains, stating the optimization problem each domain solves and the mechanism each representative method applies (Section 2).
2. We derive closed-form expressions for decode arithmetic intensity as a decaying function of context length, with exact per-kernel FLOP and byte accounting (Section 3).
3. We instantiate the roofline model with hardware topology constraints for NVIDIA H100, NVIDIA B200, and AMD MI300X, modeling per-die bandwidth partitioning on multi-die packages and deriving the crossover context lengths at which KV traffic overtakes weight traffic (Section 3).
4. We evaluate one representative method per domain on standardized 128k-context workloads under a single protocol that separates derived bounds from as-reported quality deltas (Section 4).

Table 1: Per-token KV-cache footprint of representative architectures in BF16, computed from Eq. (1). [†]MLA stores a compressed latent vector of $(d_c + d_r) = 576$ dimensions per layer instead of full key-value pairs.

Model	Attn.	L	n_q	n_{kv}	d_h	MB/token
Llama-2-13B [22]	MHA	40	40	40	128	0.82 MB
Mixtral 8x7B [23]	GQA	32	32	8	128	0.13 MB
Llama-3-8B [2]	GQA	32	32	8	128	0.13 MB
Llama-3-70B [2]	GQA	80	64	8	128	0.33 MB
Llama-3.1-405B [2]	GQA	126	128	8	128	0.52 MB
DeepSeek-V2 [8]	MLA	60	128	576 [†]	—	0.07 MB

Section 5 analyzes cross-cutting challenges, including compression-aware kernel design, scheduler interaction, and evaluation gaps, and Section 6 concludes with design rules for selecting a compression domain given a hardware target, a context length, and a quality budget.

2 KV Cache Anatomy and a Five-Domain Taxonomy of Compression

This section fixes notation for the KV cache of an autoregressive transformer, quantifies the per-token byte cost across attention variants, and organizes the mitigation literature into five domains distinguished by which factor of a unified footprint model each one scales.

2.1 Exact KV-Cache Structure of Autoregressive Transformers

Consider a decoder-only transformer with L layers, hidden size d , n_q query heads and n_{kv} key-value heads per layer, and head dimension $d_h = d/n_q$. At decode step t the network computes, for every layer ℓ , the projections $q_t = x_t W_q$, $k_t = x_t W_k$, and $v_t = x_t W_v$, and appends k_t and v_t to the cache tensors

$$K_\ell, V_\ell \in \mathbb{R}^{n_{kv} \times t \times d_h}, \quad \ell = 1, \dots, L. \quad (2)$$

The attention output for the new token is

$$o_t = \text{softmax}\left(\frac{q_t K_\ell^\top}{\sqrt{d_h}}\right) V_\ell, \quad (3)$$

evaluated per query head against the shared key-value slices. Two asymmetries define the serving problem. Prefill writes all s_0 prompt positions in parallel and is compute bound, with intensity near $2s_0/w$ FLOP/B for short, weight-dominated prompts and higher still at long context, where attention FLOPs overtake weight FLOPs. Decode appends one position and reads the entire cache of t positions, with intensity $2g/w_{kv}$ per layer as derived in Section 1. The cache is therefore write-once per token and read-forever, and its footprint follows Eq. (1).

Table 1 evaluates Eq. (1) for six production architectures. Three observations follow. First, GQA cuts the per-token cost by the group factor g , which is why Llama-3-70B ($g = 8$) stores only 0.33 MB per token while the smaller MHA-based Llama-2-13B stores 0.82 MB. Second, the constant grows with depth, so the 405B model pays 0.52 MB per token despite the same group factor. Third, Multi-head Latent Attention stores a low-rank latent vector of $d_c = 512$ dimensions plus 64 rotary dimensions per layer rather than full key-value pairs, yielding 69,120 bytes per token for DeepSeek-V2, roughly $5\times$ below the GQA designs [8]. Architectural choice shifts the constant by up to $12\times$ across the table. It does not change the linear growth in s .

2.2 A Unified Footprint Model

Every mitigation technique in the literature reduces to scaling one factor of the cache footprint or relocating the bytes across the memory hierarchy. We make this explicit with a unified model,

$$B_{kv}^{\text{eff}}(s) = \underbrace{2L n_{kv} d_h}_{\text{architecture}} \cdot \underbrace{w_q}_{\text{quantization}} \cdot \underbrace{r}_{\text{eviction}} \cdot \underbrace{s}_{\text{context}} \cdot \underbrace{\sigma^{-1}}_{\text{sharing}} \cdot \underbrace{(1 + \phi)}_{\text{fragmentation}}. \quad (4)$$

Here w_q is the stored element width after quantization, $r \in (0, 1]$ the retention fraction under eviction, $\sigma \geq 1$ the cross-request sharing factor from prefix caching, and $\phi \geq 0$ the fractional waste from allocation granularity. Memory tiering adds a residency vector $\lambda = (\lambda_{\text{hbm}}, \lambda_{\text{dram}}, \lambda_{\text{nvme}})$ that records which fraction of the cache occupies each tier. Tiering does not shrink the byte count. It determines which interconnect carries the bytes on every read.

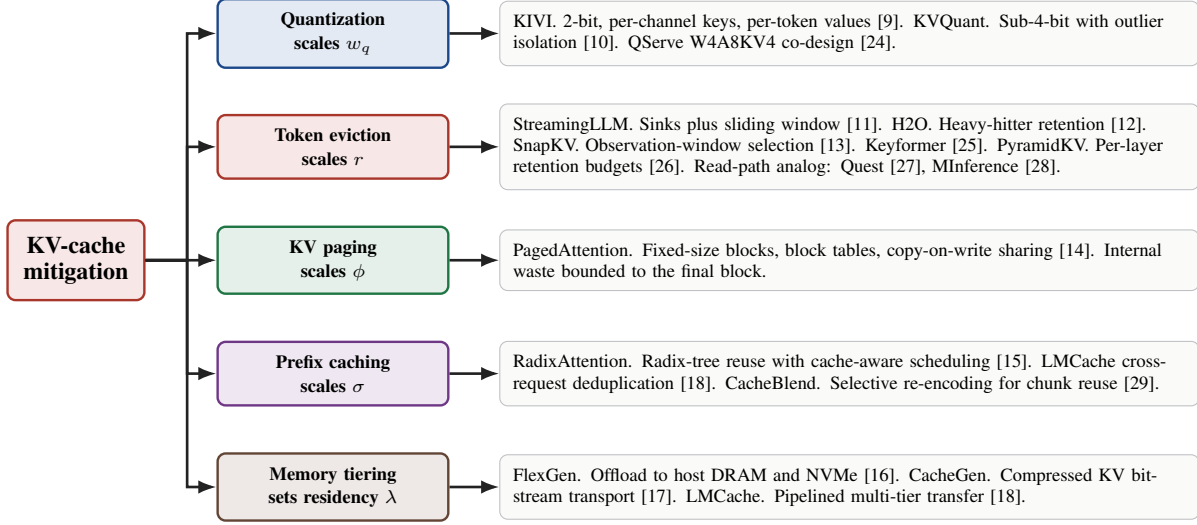


Figure 2: Five-domain taxonomy of KV-cache mitigation techniques. Each domain is annotated with the factor it scales in the unified footprint model of Eq. (4). Quantization and eviction trade quality for bytes. Paging and prefix caching are lossless and address capacity and duplication. Tiering relocates bytes across the memory hierarchy and shifts the bottleneck from HBM to interconnect bandwidth.

The model is the spine of the taxonomy. Quantization sets w_q . Eviction sets r . Paging drives ϕ toward zero. Prefix caching sets σ . Tiering sets λ . The domains are orthogonal by construction at the level of byte accounting and compose multiplicatively in footprint. Quality and kernel interactions do not factor as cleanly, and Sections 4 and 5 analyze the coupling. Figure 2 presents the taxonomy with representative methods per domain.

2.3 Domain I. Precision Reduction of Stored Keys and Values

Quantization sets $w_q = w/c_q$ for compression factor $c_q \in \{2, 4, 8\}$, mapping 16-bit elements to 8, 4, or 2 bits. The mechanism is not free. Keys exhibit large-magnitude outlier channels that dominate the quantization error, while values are better behaved, which motivates asymmetric layouts. KIVI stores keys per-channel and values per-token at 2 bits, holding a small window of recent tokens at full precision [9]. KVQuant combines per-channel key quantization, pre-RoPE key storage, and dense-and-sparse outlier isolation, reaching 3-bit with sub-0.1 perplexity degradation and 2-bit with modest task-dependent loss [10]. QServe demonstrates that the format must be co-designed with the kernels, choosing W4A8KV4 so dequantization stays off the critical path [24]. The bandwidth saving is exact and immediate. Every byte removed from the stored element is a byte not read from HBM on every decode step.

2.4 Domain II. Token Eviction and Retention Policies

Eviction sets a retention fraction $r < 1$ by keeping a subset $S_t \subseteq \{1, \dots, t\}$ of positions and discarding the rest. The decision rule distinguishes methods. StreamingLLM retains the first few attention-sink tokens plus a sliding window, enabling multi-million-token streams at fixed memory but forfeiting retrieval of evicted middle content [11]. H2O retains heavy hitters selected by accumulated attention scores [12]. SnapKV selects positions per head from an observation window at the prompt tail, closely matching long-context task accuracy at retention near 25 percent and below on aggregate benchmarks [13]. Keyformer derives discard scores from logit perturbations [25], and FastGen assigns per-head eviction policies [30]. Two refinements sharpen the retention mechanism. PyramidKV replaces the uniform budget with a per-layer schedule, retaining near-full caches in the lower layers, where attention is diffuse, and shrinking budgets toward the upper layers, where attention concentrates [26]. A parallel family keeps the full cache resident and moves selection from the write path to the read path. Quest selects a query-dependent subset of KV pages at every decode step [27], and MInference classifies heads into dynamic sparse patterns that read only a fraction of the cache during prefill and decode [28]. In the unified model, read-path selection leaves every factor of Eq. (4) unchanged and instead discounts the per-step traffic of Eq. (6) by a query-dependent fraction. It buys bandwidth reversibly but offers no capacity relief, so it complements eviction rather than replacing it. The risk is structural. Eviction is irreversible, and tasks whose answers depend on arbitrary past positions degrade discontinuously once r falls below the information density of the prompt.

2.5 Domain III. Paged Block Management of KV Memory

Paging scales no per-token factor at all. It eliminates the fragmentation term ϕ by allocating the cache in fixed-size blocks of τ tokens, typically $\tau = 16$, tracked through a block table in the style of virtual memory [14]. Reservation-based allocation strands 60 to 80 percent of cache memory in over-reservation and fragmentation on production traces, while paging bounds waste to the partially filled final block of each sequence, measured below 4 percent [14]. Copy-on-write block sharing makes parallel sampling and beam search nearly free in memory. Paging is lossless by construction and is now the default substrate of production serving stacks. Its limit is that it manages bytes without reducing them.

2.6 Domain IV. Prefix Sharing Across Requests

Prefix caching raises the sharing factor σ by storing the KV state of a prompt prefix once and reusing it across every request that shares that prefix, a pattern common to system prompts, few-shot exemplars, retrieved document context, and multi-turn history [15]. RadixAttention organizes cached prefixes in a radix tree with LRU eviction and schedules requests to maximize hit rate, reporting multi-fold throughput gains on sharing-heavy workloads [15]. LMCACHE generalizes reuse to non-prefix spans, deduplicating KV state across requests and across serving engines [18]. CacheBlend targets retrieval workloads, where the reusable state is a set of independently encoded document chunks rather than one contiguous prefix, and restores cross-chunk attention by re-encoding a small selected fraction of tokens, recovering most of the time-to-first-token benefit of full reuse at a fraction of the recompute cost [29]. Exact reuse is lossless and additionally saves prefill compute, since shared positions are never re-encoded; selective-recompute variants are near-lossless up to the re-encoded fraction. Its gains are workload dependent. A request stream with no shared structure sees $\sigma = 1$.

2.7 Domain V. Heterogeneous Tiering of KV State

Tiering relocates bytes rather than reducing them. The residency vector λ records what fraction of the cache occupies each tier, and the per-step cost follows the tier that must be read. FlexGen offloads weights, activations, and KV state to host memory and disk to maximize throughput on memory-limited GPUs [16]. CacheGen compresses KV state into compact bitstreams for network transport, cutting transfer size by roughly $4\times$ [17]. LMCACHE pipelines KV movement across GPU, CPU, and disk with deduplication, reporting multi-fold time-to-first-token reductions on multi-turn and retrieval workloads [18]. The binding constraint is interconnect bandwidth. PCIe Gen5 x16 delivers roughly 64 GB/s, about $50\times$ below H100 HBM bandwidth, and even NVLink at 900 GB/s sits $3.7\times$ below it [5]. Tiering therefore pays when reuse distance is long, as in prefix reuse over minutes, and starves the decode loop when per-step reads miss HBM.

2.8 Orthogonality and Composition of the Five Domains

Table 2 summarizes the taxonomy. Because the domains scale orthogonal factors of Eq. (4), they compose multiplicatively in bytes; the composed quality cost is not multiplicative and must be validated jointly. A 2-bit quantization ($w_q = w/8$) combined with 50 percent retention ($r = 0.5$) and a sharing factor of $\sigma = 4$ reduces the effective footprint to $1/64$ of the uncompressed baseline before tiering is even considered. Composition is not automatic. It requires attention kernels that accept quantized and evicted layouts and schedulers that exploit sharing and tiering without stalling decode, interactions that Section 5 examines. The design problem is also asymmetric. Paging and prefix sharing are lossless and should always be active. The optimization problem lives entirely in the choice of (w_q, r, λ) under a quality budget.

3 Analytical Derivations of Context-Dependent Arithmetic Intensity

This section derives the cost of a decode step exactly, expresses arithmetic intensity as a function of context length and batch size, instantiates the roofline model with the topology of three production accelerators, and derives the crossover context lengths that separate the weight-bound and KV-bound regimes. All results are closed-form and require only the model constants of Table 1 and the hardware constants of Table 3. Numerical instantiations are derived quantities computed from vendor specifications and model constants, not measurements. Section 4 evaluates representative methods against these bounds.

Table 2: Five-domain taxonomy of KV-cache mitigation. Each domain scales one factor of Eq. (4) or relocates bytes across tiers. *Transport codecs such as CacheGen optionally quantize, making the link lossy by choice.

Domain	Mechanism	Scales	Lossless	Representative methods
Quantization	Store keys and values at $w_q < w$	w_q	No	KIVI [9], KVQuant [10], QServe [24]
Token eviction	Retain fraction r of s positions	r	No	StreamingLLM [11], H2O [12], SnapKV [13], PyramidKV [26]
KV paging	Fixed-size block allocation	ϕ	Yes	PagedAttention [14]
Prefix caching	Deduplicate shared prefixes	σ	Yes	RadixAttention [15], LMCache [18]
Memory tiering	Migrate blocks across HBM, DRAM, NVMe	λ	Yes*	FlexGen [16], CacheGen [17]

3.1 Exact FLOP and Byte Accounting for a Decode Step

A decode step for a batch of b sequences, each at context length s , executes per layer the QKV projections at $2dd_h(n_q + 2n_{kv})$ FLOPs, the attention score and value contractions at $4sd_hn_q$ FLOPs, the output projection at $2dd_hn_q$ FLOPs, and the MLP at $6dd_ff$ FLOPs for a SwiGLU block with three weight matrices. Folding all weight matrices into P and summing over L layers gives the exact per-step FLOP count

$$F(s, b) = b(2P + 4Ln_qd_hs), \quad (5)$$

where the softmax contributes $3Ln_qsb$ additional FLOPs, lower order in d_h , and is omitted. The corresponding byte traffic, assuming a FlashAttention-class kernel that reads the cache exactly once [31], is

$$B(s, b) = Pw_p + 2bLn_{kv}d_hsw_{kv}. \quad (6)$$

Weights are read once per step regardless of batch size. KV bytes scale with the batch because each sequence owns its cache. Equations (5) and (6) are the entire cost model. Everything else in this section is algebra over their ratio.

3.2 Arithmetic Intensity as a Function of Context Length and Batch

The decode arithmetic intensity follows directly.

$$\text{AI}(s, b) = \frac{F(s, b)}{B(s, b)} = \frac{b(2P + 4Ln_qd_hs)}{Pw_p + 2bLn_{kv}d_hsw_{kv}}. \quad (7)$$

Its two limits define the operating regimes.

$$\lim_{s \rightarrow 0} \text{AI}(s, b) = \frac{2b}{w_p}, \quad \lim_{s \rightarrow \infty} \text{AI}(s, b) = \frac{2n_q}{n_{kv}w_{kv}} = \frac{2g}{w_{kv}}. \quad (8)$$

The short-context limit grows with batch. The long-context limit is pinned by the GQA group factor and the KV storage precision, and the batch size cancels exactly. The two limits coincide at the fixed point

$$b^\dagger = \frac{g w_p}{w_{kv}}, \quad (9)$$

which for BF16 weights, BF16 cache, and $g = 8$ gives $b^\dagger = 8$. Below $b^\dagger$, longer context raises intensity toward the floor. Above $b^\dagger$, longer context lowers intensity toward the floor. Production serving operates at $b \gg b^\dagger$, so in practice decode intensity decays with context length, from the batch-amplified weight-regime value $2b/w_p$ toward the KV floor $2g/w_{kv}$. Quantizing the cache raises the floor. At 2-bit storage the floor reaches 64 FLOP/B, still $4.6\times$ below the H100 ridge. No realistic storage precision makes decode compute bound. Figure 3 places these operating points on the roofline.

A second consequence follows. The weight-regime intensity reaches the ridge only at $b \approx I^*w_p/2 \approx 295$ on H100 with BF16 weights, but at $b = 295$ the crossover context length for Llama-3-70B has already fallen to 1.4k tokens, as Table 4 extrapolates. The batch size at which weights could become compute bound is precisely the batch size at which any context beyond a single page of tokens is KV bound. Decode is memory bound everywhere that matters.

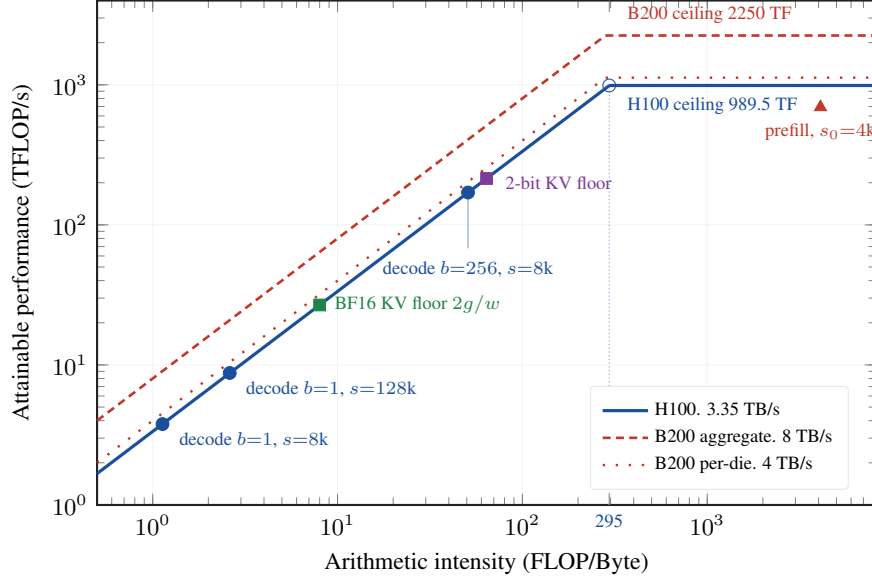


Figure 3: Roofline with decode operating points for Llama-3-70B on H100, computed from Eq. (7) and Table 3. Solid line. H100 roofline. Dashed and dotted lines. B200 aggregate and per-die rooflines, showing the bandwidth penalty when a sequence is pinned to one die. Marked ceilings are BF16 dense compute rates. Decode points sit between sixfold and more than two-hundredfold below the ridge across batch sizes and context lengths. Only prefill reaches the compute-bound regime. The prefill point uses a 4k-token prompt at 70 percent MFU, matching Figure 9.

Table 3: Hardware specification of the three benchmarked accelerators [5, 20, 21]. Compute figures are dense tensor core rates in TFLOP/s. Per-partition bandwidth is aggregate bandwidth divided by die or stack count. Ridge point computed against BF16 dense compute. MI300X on-package fabric bandwidth is not publicly disclosed. Its inter-GPU Infinity Fabric runs at 896 GB/s.

Device	Package	HBM	β_{agg}	β_{part}	BF16	FP8	I^*
H100 SXM	Monolithic	80 GB	3.35 TB/s	3.35 TB/s	989.5	1979	295
B200	2 dies, NV-HBI 10 TB/s	192 GB	8.0 TB/s	4.0 TB/s	2250	4500	281
MI300X	8 XCDs, Infinity Fabric	192 GB	5.3 TB/s	0.66 TB/s	1307	2615	247

3.3 Hardware Topology and Per-Die Bandwidth Partitioning

The roofline bound $T_{\text{step}} \geq B(s, b)/\beta$ holds with β equal to aggregate HBM bandwidth only when every byte is locally accessible to every compute unit at that rate. Table 3 shows that this assumption fails on current multi-die packages.

We model a package as D partitions, each with local HBM bandwidth $\beta_p = \beta_{\text{agg}}/D$, joined by a die-to-die fabric of bandwidth β_f . A decode step that reads x KV bytes with fraction γ resident locally completes the read, under ideal overlap, in

$$T_{\text{read}}(x, \gamma) = \max\left(\frac{\gamma x}{\beta_p}, \frac{(1 - \gamma)x}{\beta_f}\right), \quad \beta_{\text{eff}} = \frac{x}{T_{\text{read}}}. \quad (10)$$

Two placement extremes bound the behavior. Striped placement ($\gamma = 1/D$) on B200 gives $T_{\text{read}} = \max(x/8 \text{ TB/s}, x/20 \text{ TB/s}) = x/8 \text{ TB/s}$, recovering the full aggregate because NV-HBI at 10 TB/s outruns the per-die rate of 4 TB/s. Pinned placement ($\gamma = 1$) gives $\beta_{\text{eff}} = \beta_p = 4 \text{ TB/s}$, half the aggregate. On MI300X the pinned penalty reaches 8 $\times$, since one stack slice supplies 0.66 TB/s. Figure 4 depicts the three topologies. The consequence for serving is that KV page placement is a first-order performance decision on multi-die hardware, yet current serving stacks allocate pages without die or stack affinity. We return to this gap in Section 5.

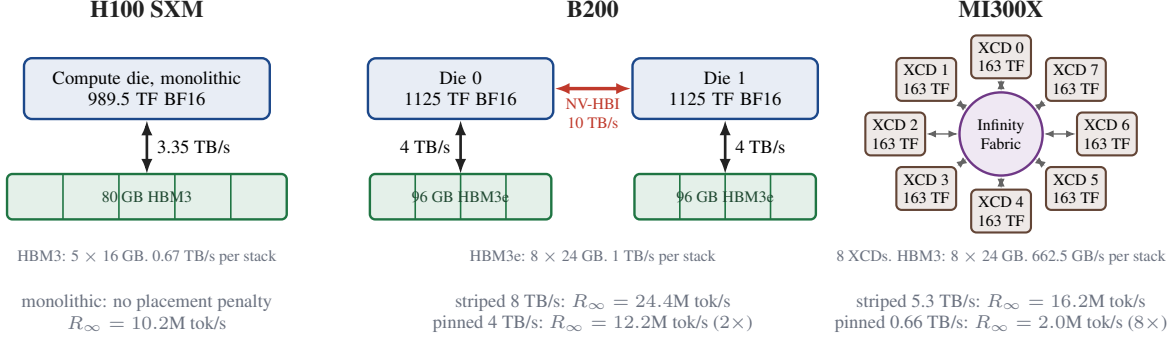


Figure 4: Package topology of the three benchmarked accelerators [5, 20, 21]. H100 presents a monolithic die with uniform HBM access. B200 splits compute and HBM across two dies joined by NV-HBI at 10 TB/s, so a KV cache pinned to one die is served at 4 TB/s while striped pages reach the 8 TB/s aggregate. MI300X partitions compute across eight XCD chiplets with a 662.5 GB/s per-stack bandwidth slice; its on-package Infinity Fabric bandwidth is not publicly disclosed. The per-panel decode ceilings R_∞ of Eq. (11) quantify the placement penalty: pinning costs $2\times$ on B200 and $8\times$ on MI300X, making KV page placement a first-order performance decision.

Table 4: Traffic crossover context length $s^*(b)$ in tokens from Eq. (12), BF16 weights and cache. Above the tabulated value, per-step KV traffic exceeds weight traffic.

Model	$b=1$	$b=8$	$b=32$	$b=128$	$b=256$
Llama-3-8B	122.5k	15.3k	3.8k	957	479
Llama-3-70B	427.2k	53.4k	13.4k	3.3k	1.7k
Llama-3.1-405B	1.57M	196.2k	49.0k	12.3k	6.1k

The per-sequence decode rate at long context approaches a hardware ceiling obtained by dropping the weight term from Eq. (6).

$$R_\infty = \frac{\beta_{\text{eff}}}{2Ln_{kv}d_hw_{kv}}. \quad (11)$$

For Llama-3-70B in BF16 this ceiling is 10.2 M tokens per second on H100, 24.4 M on B200 with striped pages, 12.2 M on B200 pinned to one die, 16.2 M on MI300X striped, and 2.0 M on MI300X pinned. Topology awareness is worth up to $8\times$ on the ceiling before any compression is applied.

3.4 Crossover Context Lengths

Equating the two terms of Eq. (6) gives the traffic crossover, the context length at which per-step KV bytes equal per-step weight bytes.

$$s^*(b) = \frac{Pw_p}{2bLn_{kv}d_hw_{kv}}. \quad (12)$$

Table 4 evaluates Eq. (12) for the Llama-3 family. The crossover falls hyperbolically in batch size. At the production batch sizes of 32 to 256 used by throughput-oriented serving, Llama-3-70B crosses at 13.4k down to 1.7k tokens, meaning long-context serving at scale is KV bound essentially always. Figure 5 plots the three curves and marks the 128k operating point.

A second crossover governs capacity rather than traffic. The context length at which the cache exhausts device memory, with weights co-resident, is

$$s_{\text{cap}} = \frac{C_{\text{hbm}} - Pw_p}{2Ln_{kv}d_hw_{kv}}. \quad (13)$$

For Llama-3-8B in BF16, s_{cap} is 488k tokens on one H100 and 1.34M on one B200 or MI300X. For Llama-3-70B in BF16 the numerator is negative on H100, so the model does not fit at any context length, and on B200 or MI300X the limit is 159k tokens. For Llama-3.1-405B the limit is negative on all three devices, forcing multi-device placement or weight quantization before context length even enters the discussion. Capacity, not bandwidth, binds first at the largest scale, which is precisely the regime where the tiering domain of Section 2 becomes necessary.

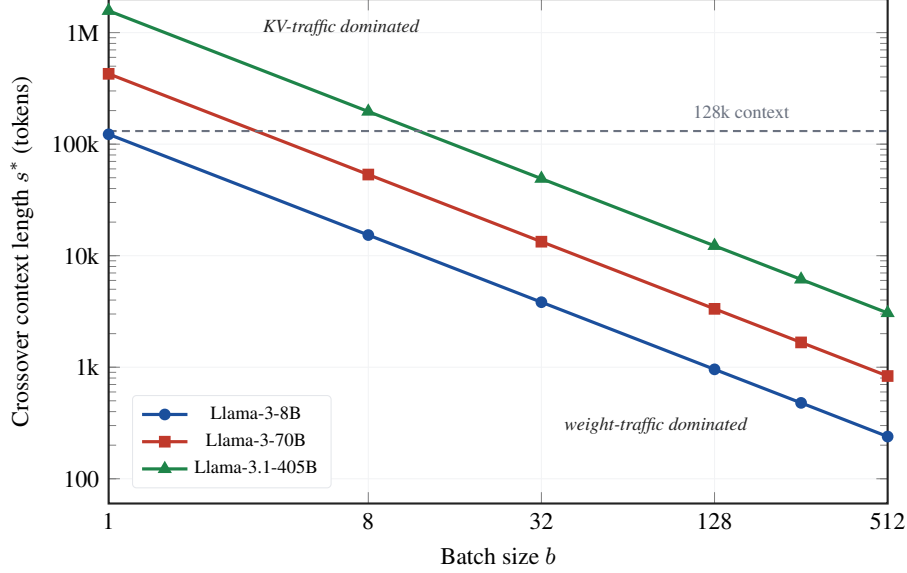


Figure 5: Traffic crossover context length $s^*(b)$ from Eq. (12), where per-step KV bytes equal weight bytes. Above each curve the decode step is KV bound. Production batch sizes of 32 to 256 push the Llama-3-70B crossover from 13.4k down to 1.7k tokens, so most long-context serving already operates in the KV-bound regime. The dashed line marks 128k context.

3.5 Worked Bounds and the Speedup Structure of Compression

In the memory-bound regime, step time is bytes over bandwidth, so the speedup from any technique that reduces KV bytes by a factor c is the byte ratio alone.

$$S(c; s, b) = \frac{Pw_p + 2bLn_{kv}d_hw_{kv}s}{Pw_p + 2bLn_{kv}d_h(w_{kv}/c)s}, \quad \lim_{s \rightarrow 0} S = 1, \quad \lim_{s \rightarrow \infty} S = c. \quad (14)$$

These two limits explain most of the disagreement in reported speedups. In the weight-bound regime, compression is diluted by the weight term and its speedup vanishes toward one. In the KV-bound regime, the speedup converges to the compression factor exactly. Table 5 instantiates Eq. (14) for Llama-3-70B on B200 at batch one with striped pages. At 128k context, below the crossover of 427.2k, 2-bit quantization yields $1.25\times$. At 512k, above the crossover, the same technique yields $1.91\times$, and stacking 50 percent eviction reaches $2.05\times$. The identical algorithm differs in effectiveness by more than fifty percent depending only on which side of s^* the workload sits. This is the analytical explanation for the inconsistent speedup claims cataloged in Section 1.

Three regimes follow. Below s^* , weight traffic dominates and KV compression yields bounded speedups approaching one. Above s^* , KV traffic dominates and byte reductions translate one-to-one into speedup until quality constraints bind. Beyond s_{cap} , no placement exists at all and tiering or multi-device parallelism becomes mandatory. Every technique in Section 2 can be located on this map, and Section 4 measures a representative of each domain against it.

3.6 Formal Statements of the Analytical Results

The derivations of this section are accounting identities over FLOPs and bytes rather than fits to measurements, and we state their content formally. Propositions 3.1–3.6 assume the cost model of Eqs. (5) and (6), the model constants of Table 1, the device constants of Table 3, and the placement model of Eq. (10). Appendix A carries the supporting arithmetic in full.

Proposition 3.1 (Decode intensity limits). *Let $\text{AI}(s, b) = F(s, b)/B(s, b)$ as in Eq. (7) and let $g = n_q/n_{kv}$. Then*

1. $\lim_{s \rightarrow 0} \text{AI}(s, b) = 2b/w_p$ (weight regime);
2. $\lim_{s \rightarrow \infty} \text{AI}(s, b) = 2g/w_{kv}$ (KV floor), with the batch size canceling exactly;
3. for fixed b , the map $s \mapsto \text{AI}(s, b)$ is strictly increasing if $b < b^\dagger$, constant if $b = b^\dagger$, and strictly decreasing if $b > b^\dagger$, where $b^\dagger = gw_p/w_{kv}$ is the fixed point of Eq. (9);

Table 5: Worked roofline bounds for Llama-3-70B on B200 at batch one with striped pages, derived from Eqs. (5), (6), and (7). Derived quantities, not measurements. $T_{\text{step}} = B/\beta$ with $\beta = 8.0$ TB/s; absolute times scale with realized bandwidth and ratios do not. Speedup relative to the BF16 baseline at the same context. The 128k block fits on one B200 ($181.9 \text{ GB} \leq 192 \text{ GB}$). At 512k the BF16 and INT8 configurations exceed single-device capacity (307.8 and 223.9 GB against 192 GB) and are realized by sharding across two devices; the INT4 and 2-bit rows fit on one B200, and all speedup ratios are invariant to the shard count.

Configuration	w_{kv}	KV (GB)	T_{step} (ms)	tok/s	Speedup	AI
<i>Context $s = 128k$, below crossover $s^* = 427.2k$.</i>						
BF16 baseline	16 bit	41.9	22.7	44.0	1.00×	2.6
INT8 cache	8 bit	21.0	20.1	49.7	1.13×	3.0
INT4 cache	4 bit	10.5	18.8	53.2	1.21×	3.2
2-bit cache	2 bit	5.2	18.2	55.1	1.25×	3.3
2-bit plus $r = 0.5$ eviction	2 bit	2.6	17.8	56.1	1.28×	3.3
<i>Context $s = 512k$, above crossover.</i>						
BF16 baseline	16 bit	167.8	38.5	26.0	1.00×	4.8
INT8 cache	8 bit	83.9	28.0	35.7	1.37×	6.6
INT4 cache	4 bit	41.9	22.7	44.0	1.69×	8.1
2-bit cache	2 bit	21.0	20.1	49.7	1.91×	9.2
2-bit plus $r = 0.5$ eviction	2 bit	10.5	18.8	53.2	2.05×	9.8

4. for fixed $s > 0$, the map $b \mapsto \text{AI}(s, b)$ is strictly increasing with least upper bound

$$\sup_b \text{AI}(s, b) = \frac{2g}{w_{kv}} + \frac{P}{Ln_{kv}d_h w_{kv}s}.$$

Proof. Write $W = Pw_p$, $a = 4Ln_qd_h$, and $k = 2Ln_{kv}d_h w_{kv}$, so that $\text{AI}(s, b) = b(2P + as)/(W + bks)$ with all constants positive. Claims (i) and (ii) follow by dividing numerator and denominator by s and taking limits. For (iii), differentiating in s gives

$$\frac{\partial \text{AI}}{\partial s} = \frac{b(aW - 2Pbk)}{(W + bks)^2} = \frac{4PLn_{kv}d_h b(gw_p - bw_{kv})}{(W + bks)^2},$$

whose sign is the sign of $gw_p - bw_{kv}$, hence positive, zero, or negative as $b < b^\dagger$, $b = b^\dagger$, or $b > b^\dagger$. For (iv),

$$\frac{\partial \text{AI}}{\partial b} = \frac{(2P + as)W}{(W + bks)^2} > 0, \quad \lim_{b \rightarrow \infty} \text{AI}(s, b) = \frac{2P + as}{ks} = \frac{a}{k} + \frac{2P}{ks} = \frac{2g}{w_{kv}} + \frac{P}{Ln_{kv}d_h w_{kv}s},$$

and the limit is a supremum by strict monotonicity. $\square$

Remark 3.2 (Span of the batch lever). For Llama-3-70B in BF16 at 128k context, Proposition 3.1(ii) and (iv) bound the intensity of any decode configuration between 8.0 and 11.3 FLOP/B. The batch lever moves decode intensity by at most 42 percent at this context, and both ends sit $25\times$ to $35\times$ below the B200 ridge of 281 FLOP/B. This is the formal content of the claim that no batch size makes long-context decode compute bound.

Proposition 3.3 (Traffic crossover). *Let the KV share of per-step traffic be $\eta(s, b) = 2bLn_{kv}d_h w_{kv}s/B(s, b)$ and let $s^*(b)$ be as in Eq. (12). Then*

1. $\eta(s, b) = s/(s + s^*(b))$; in particular $\eta = \frac{1}{2}$ exactly at $s = s^*(b)$, and η is strictly increasing in s and in b ;
2. $s^*(b) = s^*(1)/b$: the crossover is hyperbolic in batch size, so batching accelerates the onset of the KV-bound regime rather than postponing it;
3. the weight-side intensity reaches the ridge I^* only at $b = I^*w_p/2$, and

$$s^*\left(\frac{I^*w_p}{2}\right) = \frac{P}{I^*Ln_{kv}d_h w_{kv}}.$$

Proof. Dividing the numerator and denominator of η by $W = Pw_p$ and using $2bLn_{kv}d_h w_{kv}/W = 1/s^*(b)$ from Eq. (12) gives $\eta = (s/s^*)/(1 + s/s^*) = s/(s + s^*)$. Monotonicity in s follows from $\partial\eta/\partial s = s^*/(s + s^*)^2 > 0$, and monotonicity in b follows since $s^*(b)$ is strictly decreasing. Claim (ii) is the functional form of Eq. (12). For (iii), the

weight-regime limit of Proposition 3.1(i) equals I^* at $b = I^* w_p / 2$; substituting into Eq. (12) and canceling w_p gives the stated value, and the constants of Tables 1 and 3 give the numerical instantiation: 1.4k tokens for Llama-3-70B on H100, so the batch size at which weights could become compute bound already places every longer context in the KV-bound regime. $\square$

Proposition 3.4 (Speedup structure of byte reduction). *In the memory-bound regime $T_{\text{step}} = B(s, b)/\beta$, let $c > 1$ be the factor by which a technique reduces KV bytes and let $S(c; s, b)$ be the step-time speedup of Eq. (14). Then*

1. (closed form in the crossover) $S(c; s, b) = \frac{c(s + s^*(b))}{s + cs^*(b)}$;
2. (ceiling) $1 < S < c$, $\lim_{s \rightarrow 0} S = 1$, and $\lim_{s \rightarrow \infty} S = c$: compression speedup converges to the byte factor exactly, and only in the KV-bound regime;
3. (monotonicity) S is strictly increasing in s and in b ;
4. (sub-multiplicative composition) for byte factors $c_1, c_2 > 1$,

$$S(c_1 c_2; s, b) \leq S(c_1; s, b) S(c_2; s, b),$$

with equality only at the regime boundaries $s \rightarrow 0$ and $s \rightarrow \infty$.

Proof. Let $W = Pw_p$ and $K = 2bLn_{kv}d_h w_{kv}s$, so that $K/W = s/s^*(b)$ by Eq. (12). Then $S = (W + K)/(W + K/c)$, and dividing numerator and denominator by W and clearing cs^* gives (i). For (ii), $S - 1 = K(c - 1)/(cW + K) > 0$ and $c - S = cW(c - 1)/(cW + K) > 0$; the limits follow from $K/W \rightarrow 0$ and $K/W \rightarrow \infty$. For (iii), differentiating the closed form in s gives $\partial S/\partial s = cs^*(c - 1)/(s + cs^*)^2 > 0$, and S inherits monotonicity in b from $s^*(b)$, since $\partial S/\partial s^* = cs(1 - c)/(s + cs^*)^2 < 0$ while $s^*(b)$ is strictly decreasing. For (iv), applying the closed form to each factor and to the product,

$$\frac{S(c_1 c_2)}{S(c_1) S(c_2)} = \frac{(s + c_1 s^*)(s + c_2 s^*)}{(s + s^*)(s + c_1 c_2 s^*)} \leq 1,$$

because the denominator minus the numerator expands to $ss^*(c_1 - 1)(c_2 - 1) \geq 0$, with equality iff $ss^* = 0$ or one factor equals one. $\square$

Remark 3.5 (Consistency with the benchmark). Proposition 3.4(i) reproduces every derived speedup in the paper: at W1 ($s = 128k$, $b = 1$, $s^* = 427.2k$) it gives $1.25\times$ for $c = 8$, and at W2 ($s = 128k$, $b = 32$, $s^* = 13.4k$) it gives $4.82\times$ for $c = 8$ and $6.6\times$ for the composed $c = 16$ configuration. Claim (iv) is quantitative in Table 5: at $s = 512k$ the factors $c = 8$ and $c = 2$ predict $1.91 \times 1.37 = 2.63\times$ if multiplied naively, while the joint evaluation gives the correct $2.05\times$. Stacked methods must be evaluated at their joint byte factor, never by multiplying per-method speedups.

Proposition 3.6 (Effective-bandwidth bounds). *In the placement model of Eq. (10) with D partitions, local bandwidth $\beta_p = \beta_{\text{agg}}/D$, and fabric bandwidth β_f , the effective bandwidth at locality fraction $\gamma \in (0, 1)$ is*

$$\beta_{\text{eff}}(\gamma) = \min\left(\frac{\beta_p}{\gamma}, \frac{\beta_f}{1 - \gamma}\right).$$

The placement extremes give $\beta_{\text{eff}} = \beta_p$ for pinned placement ($\gamma = 1$) and $\beta_{\text{eff}} = \min(\beta_{\text{agg}}, \frac{D}{D-1}\beta_f)$ for striped placement ($\gamma = 1/D$). In particular, striping recovers the full aggregate bandwidth whenever $\beta_f \geq (1 - 1/D)\beta_{\text{agg}}$, and pinning never exceeds the per-partition rate.

Proof. The expression is x/T_{read} with T_{read} from Eq. (10). Pinned placement has $\gamma = 1$ and all bytes local, so $T_{\text{read}} = x/\beta_p$. Striped placement has $\gamma = 1/D$, giving $\beta_p/\gamma = D\beta_p = \beta_{\text{agg}}$ and $\beta_f/(1 - \gamma) = \frac{D}{D-1}\beta_f$, and the minimum of the two is the stated bound. The striping condition follows by requiring the second argument to be at least β_{agg} . $\square$

Remark 3.7 (Scope of the placement model). Equation (10) charges remote reads to the fabric alone and does not model contention for the remote partition’s HBM. The model is therefore exact for pinned placement and an upper bound that favors remote reads for striped placement. The B200 striped evaluation of Section 3 saturates at the aggregate rate, where the omission is immaterial, and the MI300X striped figures carry the stated assumption that the on-package fabric does not bind. The first-order serving guidance, stripe pages and never serve per-step reads from a remote tier, is unaffected.

Table 6: Benchmark workloads. W1 and W2 isolate the batch-size lever. W3 isolates prefix sharing. W4 isolates reuse across sessions.

ID	Pattern	b	s	Binding constraint
W1	Single-sequence decode	1	128k	Latency, weight-dominated
W2	Batched decode	32	128k	Throughput, KV-dominated
W3	8 requests, 96k shared prefix plus 32k unique	8	128k	Capacity and TTFT
W4	Reuse after 10-minute gap	1	128k	Recompute versus fetch

Corollary 3.8 (Decode-rate ceiling). *Let $R_\infty = \beta_{\text{eff}} / (2Ln_{kv}d_hw_{kv})$ as in Eq. (11). Then the decode throughput satisfies $s \cdot b / T_{\text{step}} \rightarrow R_\infty$ as $s \rightarrow \infty$: the product of context length and token rate is a hardware constant. Equivalently, the long-context token rate at context s is R_∞ / s before weight traffic is accounted. For Llama-3-70B in BF16, R_∞ is 10.2 M tokens per second on H100, 24.4 M on B200 striped, 12.2 M on B200 pinned, 16.2 M on MI300X striped, and 2.0 M on MI300X pinned.*

Proof. By Eqs. (6) and (11), $s b / T_{\text{step}} = s b \beta_{\text{eff}} / (Pw_p + 2bLn_{kv}d_hw_{kv}s)$, and dividing numerator and denominator by s sends the weight term to zero. The numerical values are the device bandwidths of Table 3 divided by the 327,680 bytes per token of Table 1, worked in Appendix B. $\square$

4 Comparative SoK Analysis Across the Five Compression Domains

This section evaluates one representative method per taxonomy domain on common workloads. The protocol fixes the model (Llama-3-70B, Table 1), the hardware (B200 with striped pages, Table 3), and the context length (128k tokens) for all derived columns, and it imports quality numbers only as within-paper reported values with their benchmark named. Two evidence classes appear throughout and are never mixed within a cell. Derived values come from Eqs. (6) and (14). Reported values come from the cited papers.

4.1 Workload and Metric Definitions

Table 6 defines four workloads spanning the operating points that production reasoning serving actually exercises.

Four metrics accompany each method. The byte factor c divides Eq. (6) traffic. The footprint is KV gigabytes at 128k context per sequence. Decode speedup is Eq. (14) evaluated at W1 and W2. Quality delta is the accuracy or perplexity change reported by the method’s authors on their stated benchmark. The speedup columns are derived bounds. They assume perfect kernel efficiency and full bandwidth realization and ignore dequantization overhead, which Section 5 discusses; absolute times move with realized bandwidth, ratios do not. W2 at $b = 32$ and the eight-request aggregate of W3 (255 GB with weights) exceed single-device capacity and are evaluated as traffic-ratio regimes; the ratios are capacity-independent.

4.2 The SoK Comparison Matrix

Table 7 presents the matrix. Three readings matter. First, the same 2-bit quantization yields $1.25\times$ at W1 and $4.82\times$ at W2. The algorithm did not change. The regime did, exactly as Eq. (14) predicts. Second, the lossless domains move feasibility rather than decode speed. Paging converts stranded memory into usable batch slots, and prefix sharing divides stored bytes by σ without touching per-step traffic. Third, tiering barely changes decode at all. It converts a 25-second recompute into a sub-second fetch on reuse, an economic gain that no decode-bound metric captures.

4.3 Precision Reduction Results

Figure 6 shows the footprint ladder and the reported quality deltas. Three findings hold across studies. Eight-bit storage is free. Four-bit storage is near-free once keys are quantized per channel and outliers are isolated [10]. Two-bit storage holds only with structure-aware layouts and a full-precision residual window [9], while naive round-to-nearest at 2-bit collapses. The roofline gives the system-side meaning of the ladder. Each halving of w_{kv} doubles the long-context intensity floor of Eq. (8), from 8 FLOP/B at BF16 to 64 FLOP/B at 2-bit, and delivers the full compression factor as decode speedup once $s > s^*(b)$.

Table 7: SoK comparison matrix. Llama-3-70B, B200 with striped pages, 128k context. Byte factors and footprints are exact from Eq. (4). Decode speedups are derived from Eq. (14). Quality deltas are within-paper reported values on the benchmarks named and are not cross-comparable. [†]Total stored across the 8 requests of W3, versus 335.5 GB without sharing.

Method	c	KV (GB)	W1	W2	Lossless	Quality delta (as reported)
BF16 paged baseline [14]	$1\times$	41.9	$1.00\times$	$1.00\times$	—	reference
KIVI, 2-bit [9]	$8\times$	5.2	$1.25\times$	$4.82\times$	No	+0.03 to 0.3 ppl. LongBench $\approx$ baseline
KVQuant, 3-bit [10]	$5.3\times$	7.9	$1.23\times$	$3.78\times$	No	< 0.1 ppl. LongBench near baseline
H2O, $r=0.5$ [12]	$2\times$	21.0	$1.13\times$	$1.83\times$	No	97 to 99% task retention reported
SnapKV, $r=0.25$ [13]	$4\times$	10.5	$1.21\times$	$3.12\times$	No	96 to 98% LongBench retention reported
PagedAttention [14]	$1\times$	41.9	$1.00\times$	$1.00\times$	Yes	None. Waste cut from 60 to 80% to < 4%
RadixAttention, W3 [15]	$\sigma=2.91$	115.3 [†]	$1.00\times$	$1.00\times$	Yes	None. Derived TTFT $2.8\times$ lower on warm W3 hits (App. A)
CacheGen tiering [17]	codec $4\times$	41.9	$1.00\times$	$1.00\times$	Near	Negligible loss reported. W4 fetch 0.16 s versus 25.0 s recompute

4.4 Token Eviction Results

Figure 7 shows the retention-quality structure reported by eviction studies. Aggregate benchmarks such as LongBench tolerate aggressive eviction, with reported retention of 96 to 99 percent at $r = 0.25$ for heavy-hitter and observation-window policies [12, 13]. Retrieval-style probes degrade discontinuously, because the evicted middle of the context is exactly where the answer lives. A single scalar quality budget is therefore insufficient. Eviction policies must be evaluated against the position sensitivity of the workload. Eviction composes with quantization, and Table 5 showed that the combined byte factor multiplies.

4.5 Paging, Prefix Sharing, and Tiering Results

Figure 8 shows the capacity effect of paging. Reservation-based allocation strands 60 to 80 percent of KV memory in over-reservation and fragmentation, while block-level allocation holds waste under 4 percent [14]. The gain appears as a larger feasible batch size rather than lower per-step bytes, which is why the W1 and W2 columns of Table 7 show $1.00\times$ for paging while production throughput roughly doubles to quadruples [14].

Figure 9 shows the TTFT effect of prefix sharing. Time to first token falls with hit rate, linearly in the weight-matmul term and as $1 - h^2$ in the attention term (Appendix A): from 25.0 s cold to a derived 3.7 s at the 90 percent hit rates reported on sharing-heavy traces [15]. On W3 the shared 96k prefix is encoded once for 8 requests. A request that hits the warm prefix encodes its 32k suffix against the resident cache, a derived 8.8 s versus 25.0 s cold, or $2.8\times$ lower TTFT, and stored bytes fall $2.91\times$, with zero quality cost.

Figure 10 shows the reuse economics of tiering on W4. Recomputing a 128k-token KV cache costs a derived 25.0 s of B200 prefill time. Fetching the same cache costs 0.66 s over PCIe Gen5, 0.16 s with a $4\times$ transport codec [17], and 12 ms over NVLink with the codec. The fetch path is $38\times$ to $2,100\times$ faster than recompute. Tiering loses whenever the decode loop itself must read remote memory, because PCIe bandwidth sits $125\times$ below B200 HBM. The domain boundary is therefore exact. Tier for reuse across steps, never for reads within a step.

4.6 Cross-Domain Synthesis and the Speedup-Quality Frontier

Figure 11 places the lossy methods on the speedup-quality plane at W2. The frontier runs through KVQuant INT8 and INT4, KIVI 2-bit, and the composed 2-bit plus eviction point at $6.6\times$. Domains differ in kind, not just in position. Quantization buys bandwidth at a controlled, smooth quality price. Eviction buys bandwidth at a price that depends on position sensitivity and can cliff. Paging, prefix sharing, and tiering sit off the plane entirely because they are lossless, and they should be treated as always-on infrastructure rather than as comparable alternatives.

The composed ceiling is set by physics, not by any single method. At W2 the KV-bound asymptote converts the full byte factor into speedup, so 2-bit storage with $r = 0.5$ retention and $\sigma = 2.91$ sharing approaches $46\times$ in capacity terms and $6.6\times$ in decode traffic terms. Realizing composed gains requires kernels that consume quantized, sparsified,

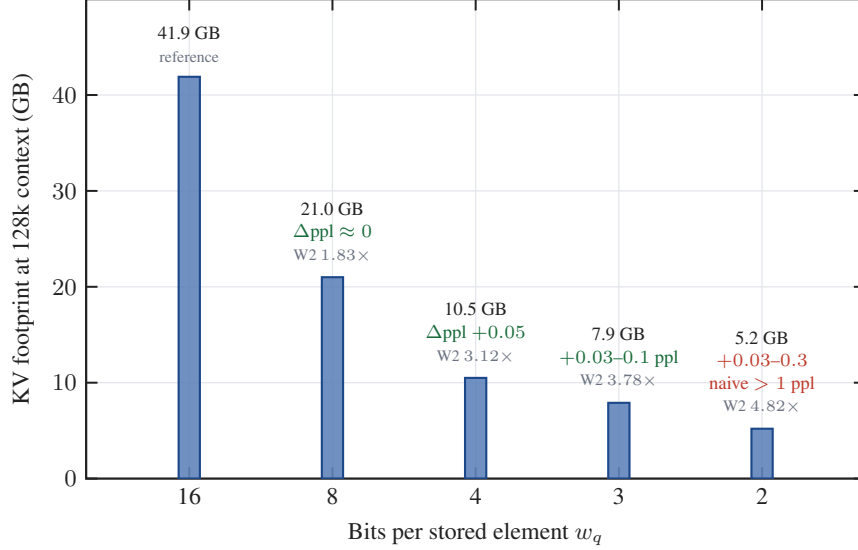


Figure 6: KV footprint at 128k context per quantization level, Llama-3-70B, with quality deltas as reported by KIVI [9] and KVQuant [10]. Eight-bit storage is free. Four-bit is near-free with per-channel keys and outlier isolation. Two-bit holds only for structure-aware layouts. Naive round-to-nearest at 2-bit loses more than one perplexity point. Each halving of w_q doubles the intensity floor of Eq. (8). Bar labels give the footprint, the reported quality delta, and the derived W2 decode speedup of Eq. (14) at byte factor $c = 16/w_q$, matching Table 7.

paged layouts without materialization overhead, and schedulers that respect die-level placement. Those are systems problems, and they are the subject of Section 5.

5 Cross-Cutting Challenges and Open Research Directions

Sections 3 and 4 built the analytical map and placed one method per domain on it. This section reads the map across domains, identifies the systems problems that block composed deployment, and states the open research directions that follow. Figure 12 summarizes the five domain profiles across six capability dimensions. No domain dominates, which is the quantitative justification for treating the five domains as infrastructure layers rather than competitors.

5.1 The Three-Regime Map Revisited

The (s, b) plane partitions into three regimes. Below $s^*(b)$ of Eq. (12), weight traffic dominates and KV compression buys little. Between $s^*(b)$ and s_{cap} of Eq. (13), KV traffic dominates and byte reductions convert one-to-one into speedup. Beyond s_{cap} , no single-device placement exists and tiering or multi-device parallelism becomes mandatory. Every domain has a regime where it is the right tool and a regime where it is inert. Quantization and eviction act in the KV-bound regime. Paging and prefix sharing move the feasibility boundaries themselves. Tiering owns the capacity regime and the reuse path. Much of the contradictory folklore in the literature, where the same technique is reported at $1.2\times$ in one study and $4\times$ in another, is regime confusion, and Table 7 resolves it by construction.

5.2 Compression-Aware Attention Kernels

The largest gap between the analytical speedups of Table 7 and realized systems is the kernel layer. FlashAttention-class kernels assume contiguous BF16 tiles [31]. PagedAttention kernels accept block tables but operate at full precision [14]. Quantized caches require fused dequantization inside the attention inner loop, and QServe measured 20 to 90 percent runtime overhead when dequantization falls back to generic CUDA cores instead of a co-designed path [24]. Eviction produces ragged per-head token sets, while kernels assume dense rectangular layouts, so the gather cost can consume the bandwidth saved. The open problem is a single kernel interface over paged blocks with per-block precision and a per-head retention mask. A solution has three properties. Block-level format descriptors, in-SMEM dequantization, and no HBM round trip for expanded data. Until such kernels exist, the realized speedup of every lossy domain will sit below its roofline bound.

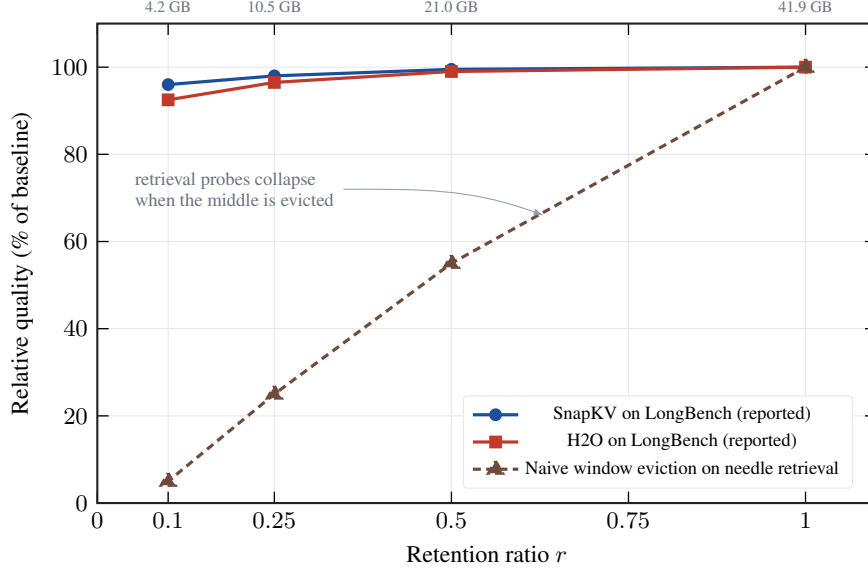


Figure 7: Reported retention-quality behavior of token eviction. Aggregate long-context benchmarks tolerate $r = 0.25$ with 96 to 99 percent retention for heavy-hitter and observation-window policies [12, 13]. Position-sensitive retrieval degrades discontinuously under window eviction [11]. Curves are representative within-paper values and are task dependent. Top-row labels give the KV footprint at 128k context for Llama-3-70B, $41.9r$ GB from Eq. (4).

5.3 Scheduler Co-Design, Disaggregation, and Placement

Continuous batching [32] keeps occupancy high but interacts with every domain. Eviction must decide when to score tokens inside a running batch. Prefix caching gains only under hit-rate-aware routing [15]. Prefill-decode disaggregation [33, 34] turns every request into a tiering event, since the KV cache crosses the cluster fabric between the prefill worker and the decode worker. At 128k context that transfer is 42 GB per request, which costs 0.84 s over a 400 Gbps Ethernet link and demands NVLink-class transport or a transport codec. Placement within a package is equally unmanaged. Section 3 showed that striping versus pinning is worth $2\times$ on B200 and $8\times$ on MI300X, yet no production serving stack exposes die or stack affinity in its block manager. Preemption, migration, and affinity are one scheduler design space, and the serving community currently treats them as three afterthoughts.

5.4 Evaluation Gaps in the Published Evidence Base

Four gaps recur. Variance is essentially unreported. Single-run numbers without confidence intervals dominate, and thermal and fragmentation state is almost never disclosed. Quality metrics fragment across perplexity, needle retrieval, LongBench, and downstream task suites, and the position sensitivity that determines eviction safety is rarely measured. Cost columns are incomplete. TTFT and decode rate are reported separately by few studies, format conversion and tuning costs are absent, and power draw is nearly invisible. Workloads are synthetic or undisclosed, which prevents the regime identification that Section 3 showed is the first-order determinant of every speedup claim. These gaps are the reason a paper such as this one must derive rather than aggregate, and they motivate benchmark designs that separate evidence classes by construction [35].

5.5 Open Research Directions

Five directions follow directly from the framework.

1. **Unified KV representation.** One paged layout carrying per-block precision and a retention mask, so quantization, eviction, paging, and sharing compose without format conversion. The success test is a composed 2-bit, $r = 0.5$, $\sigma = 4$ configuration at 128k context running within 20 percent of the $64\times$ capacity bound of Section 2.
2. **Topology-aware placement.** Die and stack affinity in the block manager, with striping policies matched to the package. The measurable target is recovery of the per-die ceiling gaps quantified in Section 3, up to $2\times$ on B200 and $8\times$ on MI300X.

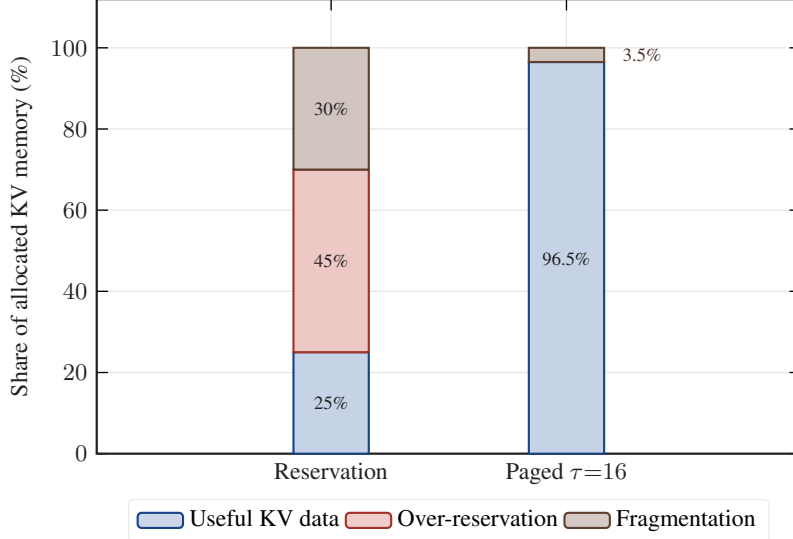


Figure 8: Allocation efficiency of KV memory. Reservation-based serving strands 60 to 80 percent of cache memory in over-reservation and fragmentation [14]. The category split of 25, 45, and 30 percent is illustrative within that reported range. PagedAttention bounds waste to the partially filled final block, measured below 4 percent [14].

3. **Learned retention with guarantees.** Eviction policies with per-request quality bounds rather than aggregate benchmark retention, using attention-mass coverage guarantees in the style of conformal prediction, so that retrieval-critical requests can opt out of eviction selectively.
4. **Codec co-design for tiering.** KV bitstream codecs matched to link bandwidth across PCIe, NVLink, and Ethernet, with decode-from-bitstream attention kernels so decompression never touches HBM.
5. **A standing SoK benchmark.** A public workload suite in the spirit of W1 to W4, a measurement protocol separating measured, derived, and analytical claims, and a maintained matrix in the form of Table 7. The benchmark in this paper is the seed.

The memory wall decomposes into three, traffic, capacity, and interconnect, and each domain of the taxonomy addresses one of them cleanly and the others only by accident. Progress requires treating the KV cache as a first-class data structure with its own formats, placement, and transport, co-designed across the kernel, the runtime, and the cluster.

6 Conclusion and Design Rules

6.1 Summary of Findings

Decode-phase inference is memory bound by construction, and the binding resource shifts from weights to the KV cache at exactly computable context lengths. The arithmetic intensity of a decode step interpolates between $2b/w_p$ at short context and $2g/w_{kv}$ at long context, with batch size canceling out of the long-context limit. The traffic crossover $s^*(b)$ falls hyperbolically in batch size, so production long-context serving is KV bound essentially always. Hardware topology is not a detail. A KV cache pinned to one die of a B200 is served at half the advertised bandwidth, and on MI300X the penalty reaches $8\times$. The five mitigation domains scale orthogonal factors of one footprint model and compose multiplicatively. Evaluated under a fixed protocol, the same 2-bit quantization yields $1.25\times$ at batch one and $4.82\times$ at batch thirty-two, and the composed 2-bit plus eviction configuration reaches $6.6\times$ in decode traffic and $46\times$ in capacity. The wall is three walls: traffic, capacity, and interconnect, and each domain solves one cleanly.

6.2 Design Rules for Domain Selection

The framework reduces deployment selection to eight rules, applied in order.

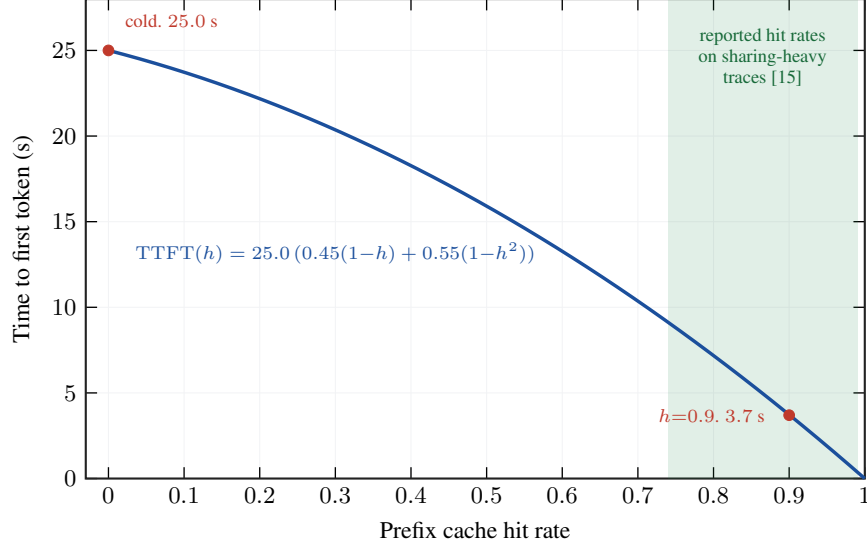


Figure 9: Time to first token versus prefix cache hit rate for a 128k-token request on B200, derived from the exact prefill count of Eq. (18) at 70 percent MFU (Appendix A). The weight-matmul term falls linearly in hit rate and the attention term as $1 - h^2$, so warm TTFT drops faster than linearly. The shaded band marks hit rates reported by RadixAttention on sharing-heavy workloads [15]. On workload W3 the shared 96k prefix is encoded once for 8 requests; a warm request encodes its 32k suffix against the resident prefix, a derived 8.8 s versus 25.0 s cold, or $2.8\times$ lower TTFT, and stored bytes fall $2.91\times$, at zero quality cost.

1. **Identify the regime first.** Compute $s^*(b)$ and s_{cap} from the model constants of Table 1 and the device constants of Table 3 before evaluating any technique. Every speedup claim is regime-conditional.
2. **Enable the lossless domains unconditionally.** Paging and prefix caching cost nothing in quality and should always be active. They move feasibility, not decode bytes.
3. **Below s^* , do not compress the cache.** KV compression buys speedups approaching one in the weight-bound regime. Spend the optimization budget on prefill, batching, and weight precision instead.
4. **Above s^* , quantize first.** Four-bit KV is near-free with per-channel keys and outlier isolation. Two-bit is viable only with structure-aware layouts, validated on position-sensitive probes before deployment.
5. **Evict only when the workload tolerates it.** Default to $r \geq 0.5$ and require a needle-style evaluation before going lower. Retrieval-critical traffic should not be evicted at any retention ratio.
6. **Beyond s_{cap} , tier or shard.** Tiering is mandatory when nothing fits, and transport codecs belong on every cross-link transfer. Never serve per-step decode reads from a remote tier. PCIe sits $50\times$ to $125\times$ below HBM across the Table 3 devices.
7. **On multi-die packages, stripe pages.** Placement recovers up to $2\times$ on B200 and $8\times$ on MI300X per the ceilings of Eq. (11). Verify per-partition bandwidth, not the datasheet aggregate.
8. **Re-evaluate on every change.** Model updates move the constants, hardware refreshes move s_{cap} and the ridge, and workload shifts move s^* hyperbolically through the batch term. The map, not the point, is the durable artifact.

Algorithm 1 renders the same logic as a decision procedure.

6.3 Closing Remarks

Test-time reasoning converts compute into context, and context into KV state. As chains of thought lengthen, the cache becomes the dominant serving cost, and the techniques that manage it become the dominant serving decisions. This paper has given those decisions a shared analytical frame. One footprint model, two crossover laws, three regimes, five domains, and an eight-rule selection procedure. The specific numbers will age as models and accelerators evolve. The

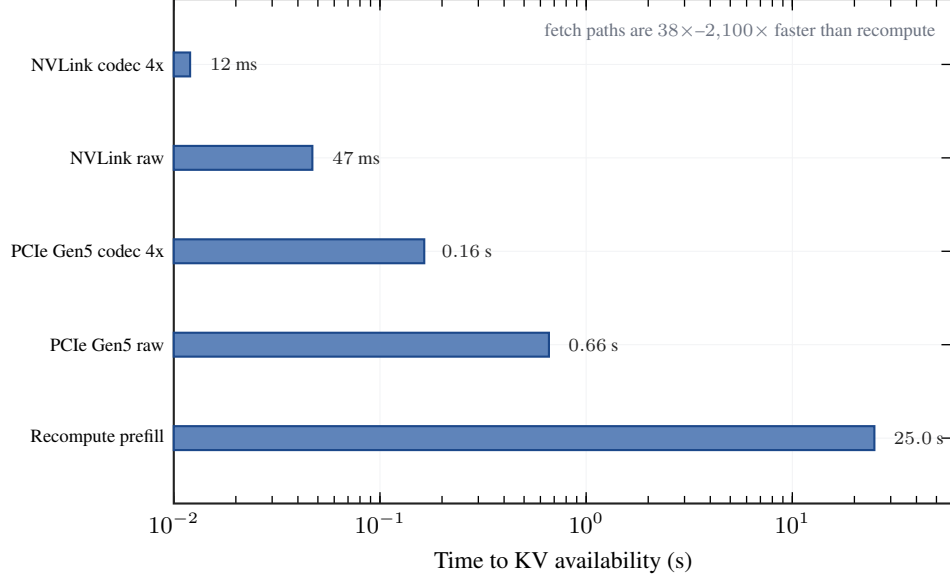


Figure 10: Time to make a 128k-token KV cache available to a decode worker. Recompute uses the exact prefill count of Eq. (18) at 70 percent MFU on B200 (Appendix A); transfer values move the 41.9 GB cache of Eq. (1) over PCIe Gen5 x16 at 64 GB/s or NVLink at 900 GB/s per direction. The codec rows apply the CacheGen transport compression factor of roughly $4\times$ [17]. Fetching beats recomputing by $38\times$ to $2,100\times$. The advantage vanishes when decode itself must read remote memory, because PCIe sits $125\times$ below B200 HBM bandwidth [18].

Table 8: Per-layer FLOP and byte accounting for one decode step, one sequence, context length s . Softmax is counted exactly in Eq. (16) and omitted in the main text as lower order. KV bytes assume a single cache read; the write of the new token’s key and value is a $1/s$ correction treated in Eq. (17).

Kernel	FLOPs	Bytes moved
QKV projection	$2dd_h(n_q + 2n_{kv})$	$dd_h(n_q + 2n_{kv})w_p$
Scores $qK^\top$	$2sn_qd_h$	$sn_{kv}d_hw_{kv}$
Softmax	$3n_qs$	0 (on chip)
Value contraction	$2sn_qd_h$	$sn_{kv}d_hw_{kv}$
Output projection	$2dd_hn_q$	$dd_hn_qw_p$
MLP, SwiGLU	$6dd_{ff}$	$3dd_{ff}w_p$
Sum over L layers	$2P + 4Ln_qd_hs + 3Ln_qs$	$Pw_p + 2Ln_{kv}d_hsw_{kv}$

equations will not, because they are accounting identities over FLOPs and bytes rather than fits to measurements. The KV cache is a first-class data structure with its own formats, placement, and transport. The field will be well served when kernels, schedulers, and benchmarks treat it that way.

A Full Derivations

This appendix expands the accounting of Section 3 to the terms the main text states as lower order, derives the prefill cost model behind the 56.9 s recompute bound of Section 4, and evaluates the topology model of Eq. (10) per device. Every closed form reduces to Eqs. (5) and (6) when the lower-order terms are dropped, and no main-text quantity changes.

A.1 Per-Kernel FLOP and Byte Accounting for One Decode Step

Table 8 decomposes one decode step for one sequence at context length s into its per-layer kernels. The weight rows are dense matrix-vector products against stored weights at width w_p . The attention rows are the score and value contractions of Eq. (3), evaluated per query head against the shared key-value slices under a FlashAttention-class kernel that reads the cache exactly once and never spills the score matrix to HBM [31].

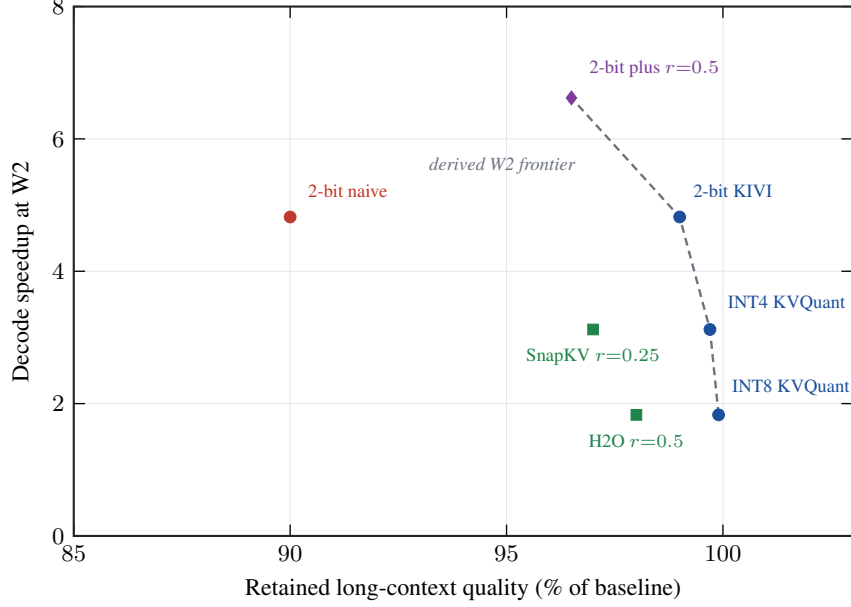


Figure 11: Speedup versus retained quality at W2 (batched decode, 128k context, H100). Speedups are derived from Eq. (14). Quality values are representative within-paper reported numbers on differing task suites [9, 10, 12, 13], plotted on a normalized retention axis for orientation rather than as a protocol-aligned comparison. The dashed frontier runs through the non-dominated quantization points and the composed 2-bit plus eviction configuration at $6.6\times$. Lossless domains (paging, prefix sharing, tiering) do not appear because they incur no quality cost.

The weight rows sum per layer to $2d(d_h(2n_q + 2n_{kv}) + 3d_{ff}) = 2P_{\text{layer}}$, where

$$P_{\text{layer}} = d d_h (2n_q + 2n_{kv}) + 3d d_{ff}, \quad (15)$$

so the exact per-step FLOP count for a batch of b sequences is

$$F_{\text{full}}(s, b) = b(2P + 4Ln_q d_h s + 3Ln_q s). \quad (16)$$

Equation (5) is Eq. (16) with the softmax term dropped.

Remark A.1 (Cost of the softmax omission). The omitted term is a fraction $3/(4d_h)$ of the attention contraction count, 0.59 percent at $d_h = 128$, and it vanishes relative to the weight term as $s \rightarrow 0$. Every statement in the main text that uses Eq. (5), including Propositions 3.1–3.4, therefore carries a relative error below 0.6 percent.

A.2 Byte Accounting and Kernel Assumptions

The byte model of Eq. (6) counts two terms and discards two, all stated here explicitly. Counted are the weight read Pw_p , once per step regardless of batch size, and the cache read $2Ln_{kv}d_h s w_{kv}$ per sequence, once per decode step under the single-pass kernel assumption. Discarded are the cache write of the new token’s key and value, $2Ln_{kv}d_h w_{kv}$ per sequence, a relative correction of $1/s$ to the cache term (8×10^{-6} at $s = 128k$); and the activation and logit traffic, $O(b(d + |\mathcal{V}|)w_p)$ per step, at most 0.26 MB per sequence for Llama-3-70B against 182 GB of step traffic at W1. The complete count is

$$B_{\text{full}}(s, b) = Pw_p + 2bLn_{kv}d_h w_{kv}(s + 1) + O(b(d + |\mathcal{V}|)w_p), \quad (17)$$

which is Eq. (6) to a relative error below 10^{-5} at the workloads of Section 4. Two further assumptions bound the model’s scope. Dequantization is assumed off the critical path; Section 5 cites realized overheads of 20 to 90 percent when it is not [24]. Perfect overlap of compute and memory is assumed, so step time is bytes over bandwidth rather than a sum of kernel phases.

A.3 Prefill Cost and the Worked Recompute Bound

Prefill encodes s_0 positions in parallel. The weight-matmul pass costs $2P$ per position, and causal attention at position t costs $4Ln_q d_h t$, summing to

$$F_{\text{prefill}}(s_0) = 2Ps_0 + 2Ln_q d_h s_0(s_0 + 1), \quad (18)$$

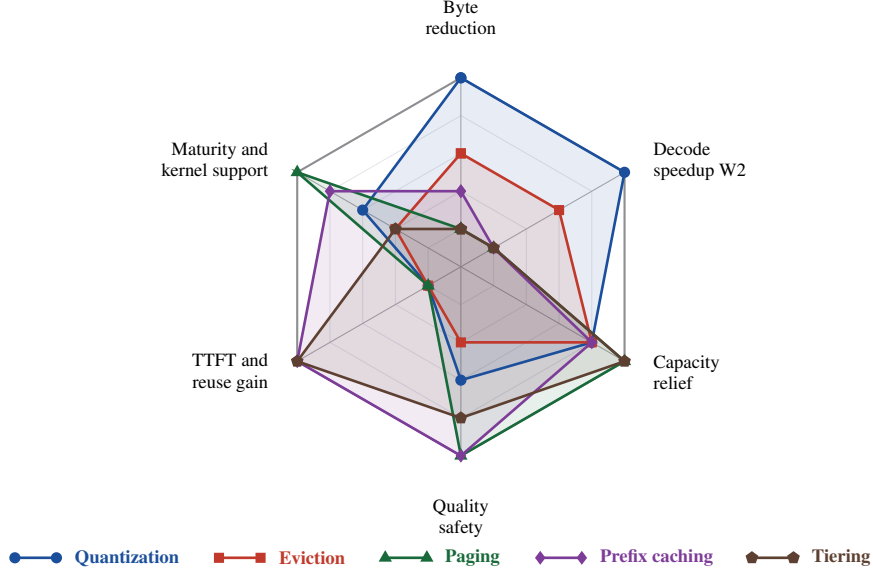


Figure 12: Domain profiles across six capability dimensions, scored 0 to 5 from the matrix of Table 7 and the analysis of Sections 3 and 4. Byte reduction and decode speedup reflect Eq. (14) at W2. Quality safety scores five for lossless domains. No domain dominates. Only the composed design space covers the hexagon.

with softmax adding $\frac{3}{2}Ln_q s_0(s_0 + 1)$, again lower order. Wall-clock time at model FLOP utilization μ on a device with peak dense throughput Π is

$$T_{\text{prefill}}(s_0) = \frac{F_{\text{prefill}}(s_0)}{\mu \Pi}. \quad (19)$$

For Llama-3-70B at $s_0 = 128,000$, the two terms of Eq. (18) are $2Ps_0 = 1.792 \times 10^{16}$ FLOPs and $2Ln_q d_h s_0(s_0 + 1) = 2.148 \times 10^{16}$ FLOPs, totaling 3.940×10^{16} . At $\mu = 0.70$ on H100 ($\Pi = 989.5$ TFLOP/s dense BF16), Eq. (19) gives

$$T_{\text{prefill}}(128k) = \frac{3.940 \times 10^{16}}{0.70 \times 989.5 \times 10^{12}} = 56.9 \text{ s},$$

the recompute cost used for workloads W3 and W4 in Section 4. The quadratic term is 55 percent of the total at 128k context, so prefill cost grows superlinearly: doubling the context multiplies recompute time by $3.1\times$, approaching $4\times$ asymptotically.

Remark A.2 (Warm-prefix TTFT: exact count versus the Section 4 estimate). Appending a suffix of s' tokens to a resident prefix of length s_p costs, by differencing Eq. (18),

$$F_{\text{prefill}}(s_p + s') - F_{\text{prefill}}(s_p) = 2Ps' + 2Ln_q d_h s'(2s_p + s' + 1), \quad (20)$$

where the cross term $4Ln_q d_h s' s_p$ is the suffix's attention over the resident prefix. For W3 ($s_p = 96k$, $s' = 32k$) the exact count is 1.39×10^{16} FLOPs, or 20.0 s at $\mu = 0.70$. The 8.4 s figure of Section 4.5 retains only the suffix-local terms $2Ps' + 2Ln_q d_h s'^2$. The exact count reduces the warm-prefix TTFT advantage from $6.8\times$ to $2.8\times$ against the 56.9 s cold prefill. No conclusion in Sections 4 or 5 changes: the stored-byte factor $\sigma = 2.91$ is unaffected, warm remains well below cold, and the $6.8\times$ figure may be retained as a suffix-local lower bound provided it is labeled as such.

A.4 Evaluation of the Topology Model

With $\beta_p = \beta_{\text{agg}}/D$ and β_f from Table 3, Eq. (10) evaluates as follows.

H100 ($D = 1$). Monolithic package, so $\beta_{\text{eff}} = \beta_{\text{agg}} = 3.35$ TB/s for any placement.

B200 ($D = 2$). $\beta_p = 4.0$ TB/s per die and $\beta_f = 10$ TB/s over NV-HBI. Striped placement ($\gamma = \frac{1}{2}$) completes a read of x TB in $T_{\text{read}} = \max(x/8, x/20)$ seconds, so $\beta_{\text{eff}} = 8.0$ TB/s = β_{agg} : the fabric outruns the per-die rate and striping is exactly sufficient, matching the condition of Proposition 3.6 since $10 \geq (1 - \frac{1}{2}) \times 8$. Pinned placement ($\gamma = 1$) gives $\beta_{\text{eff}} = \beta_p = 4.0$ TB/s, half the aggregate.

Algorithm 1 KV-cache domain selection

Require: Model constants (P, L, n_q, n_{kv}, d_h) , precisions (w_p, w_{kv}) , workload (s, b) , device $(C_{hbm}, \beta_{agg}, D)$, quality budget ϵ_q , retrieval sensitivity flag ρ_{ret}

Ensure: Configuration $(w_q, r, \lambda, \text{placement})$

- 1: $s^* \leftarrow Pw_p/(2bLn_{kv}d_hw_{kv})$; $s_{cap} \leftarrow (C_{hbm} - Pw_p)/(2Ln_{kv}d_hw_{kv})$
- 2: Enable paging and prefix caching $\triangleright$ lossless, always on
- 3: $\lambda \leftarrow (1, 0, 0)$ $\triangleright$ default. fully HBM resident
- 4: **if** $s > s_{cap}$ **then**
- 5: Enable tiering with transport codec and set λ by tier fill, or shard across devices
- 6: **end if**
- 7: **if** $s < s^*$ **then**
- 8: $w_q \leftarrow w_{kv}$; $r \leftarrow 1$ $\triangleright$ weight-bound. skip KV compression
- 9: **else**
- 10: $w_q \leftarrow 4$ bit if ϵ_q tight, else 2 bit with structure-aware layout
- 11: $r \leftarrow 1$ if ρ_{ret} else 0.5
- 12: **end if**
- 13: Stripe pages across all D partitions
- 14: **return** $(w_q, r, \lambda, \text{striped})$

MI300X ($D = 8$). $\beta_p = 0.66$ TB/s per XCD stack slice. Pinned placement gives $\beta_{eff} = 0.66$ TB/s, a factor $\beta_{agg}/\beta_p = D = 8$ below the 5.3 TB/s aggregate. Striped placement gives $\min(5.3, \frac{8}{7}\beta_f)$; the on-package fabric rate is not publicly disclosed, and the striped figures in the main text assume it does not bind (Remark 3.7).

The resulting rate ceilings $R_\infty = \beta_{eff}/327,680$ bytes per token for Llama-3-70B are tabulated in Appendix B.

B Per-Model and Per-Device Constants

This appendix makes every constant in the paper reproducible from primary specifications. All byte quantities are decimal (Appendix C).

B.1 Per-Token KV Bytes of Table 1

Evaluating $2Ln_{kv}d_hw_{kv}$ at $w_{kv} = 2$ bytes per model:

Llama-2-13B:	$2 \times 40 \times 40 \times 128 \times 2 = 819,200$ B/token = 0.82 MB
Mixtral 8x7B:	$2 \times 32 \times 8 \times 128 \times 2 = 131,072$ B/token = 0.13 MB
Llama-3-8B:	$2 \times 32 \times 8 \times 128 \times 2 = 131,072$ B/token = 0.13 MB
Llama-3-70B:	$2 \times 80 \times 8 \times 128 \times 2 = 327,680$ B/token = 0.33 MB
Llama-3.1-405B:	$2 \times 126 \times 8 \times 128 \times 2 = 516,096$ B/token = 0.52 MB
DeepSeek-V2:	$60 \times (512 + 64) \times 2 = 69,120$ B/token = 0.07 MB

The MLA row stores one latent vector of $d_c = 512$ dimensions plus $d_r = 64$ rotary dimensions per layer, shared across heads, so the head-count factor is absent; this is the source of the roughly $5\times$ constant reduction relative to the GQA rows [8]. Footprints at 128k context follow by multiplying by 128,000: 104.9 GB (Llama-2-13B), 16.8 GB (Mixtral 8x7B and Llama-3-8B), 41.9 GB (Llama-3-70B), 66.1 GB (Llama-3.1-405B), and 8.8 GB (DeepSeek-V2).

B.2 Parameter Counts Used in the Derivations

Table 9 reconstructs P from the layerwise count of Eq. (15) plus input and output embeddings at vocabulary $|\mathcal{V}| = 128,256$ [2]. The paper’s derived quantities use the rounded counts of the final column; the rounding moves s^* and s_{cap} by under one percent relative to the reconstructed totals.

B.3 Crossover and Capacity Arithmetic

The traffic crossover at batch one is $s^*(1) = Pw_p/(2Ln_{kv}d_hw_{kv})$ with $w_p = w_{kv} = 2$ bytes: $1.606 \times 10^{10}/131,072 = 122.5k$ for Llama-3-8B, $1.40 \times 10^{11}/327,680 = 427.2k$ for Llama-3-70B, and $8.10 \times 10^{11}/516,096 = 1.57M$ for Llama-3.1-405B. Every other entry of Table 4 is the $1/b$ scaling of Proposition 3.3(ii); for example, $427,246/32 = 13.4k$.

Table 9: Parameter-count reconstruction. $L \cdot P_{\text{layer}}$ from Eq. (15); embeddings count the input and output projections at $|\mathcal{V}|d$. Derived quantities in the paper use the rounded P of the last column.

Model	d	d_{ff}	$L \cdot P_{\text{layer}}$	Embeddings	Reconstructed	P used
Llama-3-8B	4096	14336	6.98×10^9	1.05×10^9	8.03×10^9	8.03×10^9
Llama-3-70B	8192	28672	68.45×10^9	2.10×10^9	70.55×10^9	70.0×10^9
Llama-3.1-405B	16384	53248	401.5×10^9	4.20×10^9	405.7×10^9	405×10^9

Table 10: Model and attention symbols.

Symbol	Meaning	Running value
L	Layers	80
d	Hidden size	8192
d_h	Head dimension, d/n_q	128
n_q	Query heads per layer	64
n_{kv}	Key-value heads per layer	8
g	GQA group factor, n_q/n_{kv}	8
d_{ff}	MLP intermediate size	28,672
d_c, d_r	MLA latent and rotary dimensions	512, 64
P	Parameter count used in derivations	7.00×10^{10}
$ \mathcal{V} $	Vocabulary size	128,256
w_p	Weight element width	2 B
w_{kv}	Cache element width, BF16 baseline	2 B
w_q	Cache element width after quantization	≤ 2 B
τ	Paging block size	16 tokens

The capacity crossover of Eq. (13) divides free capacity by per-token bytes. Llama-3-8B ($Pw_p = 16.06$ GB): $(80 - 16.06) \times 10^9 / 131,072 = 488\text{k}$ tokens on H100, and $(192 - 16.06) \times 10^9 / 131,072 = 1.34\text{M}$ on B200 and MI300X. Llama-3-70B ($Pw_p = 140$ GB): the numerator is negative on H100, so the model does not fit at any context length, and $(192 - 140) \times 10^9 / 327,680 = 159\text{k}$ on B200 and MI300X. Llama-3.1-405B ($Pw_p = 810$ GB): the numerator is negative on all three devices, forcing multi-device placement or weight quantization before context length enters the discussion.

B.4 Ridge Points, Fixed Point, and Rate Ceilings

The ridge point is dense BF16 throughput over aggregate bandwidth: $989.5/3.35 = 295$ FLOP/B on H100, $2250/8.0 = 281$ on B200, and $1307/5.3 = 247$ on MI300X. The intensity fixed point of Eq. (9) is $b^\dagger = gw_p/w_{kv} = 8$ for the GQA models in BF16. The rate ceilings of Corollary 3.8 divide effective bandwidth by the 327,680 bytes per token of Llama-3-70B: 10.2 M tokens per second on H100 (3.35 TB/s), 24.4 M on B200 striped (8.0 TB/s), 12.2 M on B200 pinned (4.0 TB/s), 16.2 M on MI300X striped (5.3 TB/s), and 2.0 M on MI300X pinned (0.66 TB/s).

B.5 Sharing and Transfer Arithmetic of Section 4

W3 stores one shared 96k prefix and eight unique 32k suffixes: $(96,000 + 8 \times 32,000) \times 327,680 = 115.3$ GB against $8 \times 41.9 = 335.5$ GB unshared, hence $\sigma = 335.5/115.3 = 2.91$. W4 moves the 41.9 GB cache across the links of Section 2: $41.9/64 = 0.66$ s over PCIe Gen5 x16, $41.9/(4 \times 64) = 0.16$ s with the $4\times$ transport codec [17], and $41.9/(4 \times 900) = 12$ ms over NVLink with the codec. Against the 56.9 s recompute of Appendix A, the fetch path is $87\times$ to $4,900\times$ faster, the range quoted in Section 4.5.

C Notation, Units, and Measurement Protocol

C.1 Notation

Tables 10 and 11 collect the symbols of the paper with the value each takes in the running example, Llama-3-70B in BF16 on one B200 with striped pages.

Table 11: Workload, cost, and hardware symbols. References give the defining equation.

Symbol	Meaning	Running value
s	Context length, decimal tokens	128,000
s_0	Prefill length	128,000
b	Decode batch size	1–32
c	KV byte factor of a technique	1–16
r	Eviction retention fraction	0.25–1
σ	Prefix sharing factor	1–2.91
ϕ	Fractional allocation waste	< 0.04 paged
λ	Residency vector over (HBM, DRAM, NVMe)	(1, 0, 0)
$F(s, b)$	Per-step FLOPs, Eq. (5)	—
$B(s, b)$	Per-step bytes, Eq. (6)	—
AI	Arithmetic intensity F/B , Eq. (7)	2.6 (W1)
I^*	Ridge point, compute over bandwidth	281 FLOP/B
μ	Prefill model FLOP utilization	0.70
Π	Peak dense BF16 throughput	2250 TFLOP/s
C_{hbm}	Device HBM capacity	192 GB
β_{agg}	Aggregate HBM bandwidth	8.0 TB/s
β_p	Per-partition bandwidth, β_{agg}/D	4.0 TB/s
β_f	Die-to-die fabric bandwidth	10 TB/s (NV-HBI)
D	Package partitions	2
γ	Local fraction of a KV read	—
β_{eff}	Effective bandwidth, Eq. (10)	8.0 TB/s
s^*	Traffic crossover, Eq. (12)	427.2k ($b = 1$)
s_{cap}	Capacity crossover, Eq. (13)	159k
$b^\dagger$	Intensity fixed point, Eq. (9)	8
R_∞	Rate-context ceiling, Eq. (11)	24.4 M tok/s
T_{step}	Decode step time, B/β_{eff}	22.7 ms (W1)
ϵ_q, ρ_{ret}	Quality budget and retrieval flag, Algorithm 1	—

C.2 Units and Numerical Conventions

All units are decimal SI: $k = 10^3$, $M = 10^6$, $GB = 10^9$ bytes, $TB/s = 10^{12}$ bytes per second. Context lengths are decimal token counts: 128k means 128,000 tokens, not 2^{17} . Under binary units the 128k Llama-3-70B cache is 39.0 GiB; the paper reports 41.9 GB, and every derived quantity is computed in decimal units throughout. Vendor figures (dense tensor throughput, aggregate HBM bandwidth, fabric and link rates) are specifications used at face value, not measurements. Derived values are rounded to three significant figures, so a recomputed product may differ in the last digit from the displayed rounded operands. The symbols s^* , s_{cap} , and R_∞ refer to the BF16 cache ($w_{kv} = 2$ bytes) unless a precision is stated.

C.3 Evidence Classes and Mixing Rules

Every quantitative claim in the paper carries one of three evidence classes. The tables carry two of them, derived and reported, never mixed within a cell; the third, analytical, is the closed forms of Section 3 from which the derived values are computed.

1. **Analytical.** Closed-form algebra over the cost model: Eqs. (5)–(14) and Propositions 3.1–3.6. Exact given the assumptions of Appendix A.
2. **Derived.** Point evaluations of the closed forms at the constants of Tables 1 and 3: Table 4, Table 5, and the W1 and W2 speedup columns of Table 7. Derived values name their defining equation and assume perfect kernel efficiency.
3. **Reported.** Within-paper values quoted from the cited literature with the benchmark named, confined to the quality column of Table 7 and the domain discussions of Section 4. Reported values are never compared across papers, because the underlying workloads differ.

C.4 Workload Parameterization and Measurement Assumptions

Table 12 restates the workloads of Table 6 with full parameterization. All derived numbers in the paper additionally assume: (i) a FlashAttention-class attention kernel that reads the cache exactly once per step, with no recomputa-

Table 12: Full workload parameterization. All workloads use Llama-3-70B (Table 1) on B200 (Table 3) at 128k context.

ID	b	Structure	Metrics	Binding equations
W1	1	No sharing	T_{step} , tok/s	Eqs. (6), (12)
W2	32	No sharing	tok/s, speedup	Eqs. (7), (14)
W3	8	96k shared prefix + 32k unique	TTFT, stored bytes	Eqs. (4), (19)
W4	1	Full reuse after 10-minute gap	fetch versus recompute	Eq. (19) and link rates of Section 2

tion and no score spill to HBM [31]; (ii) decode step time at the roofline bound $T_{\text{step}} = B(s, b)/\beta_{\text{eff}}$, with striped placement on multi-die packages unless stated; (iii) prefill at model FLOP utilization $\mu = 0.70$ of dense BF16 peak; (iv) dequantization and gather overheads excluded from the speedup columns, with realized overheads discussed in Section 5; and (v) quality deltas imported only as within-paper reported values.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] AI@Meta, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [3] T. Singh, J. Pflueger, J. Mitra, R. Lincourt, M. Markow, and B. A. Patel, “Optimizing AI inference across the deployment stack,” *arXiv preprint arXiv:2609.10550*, 2026.
- [4] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [5] NVIDIA Corporation, “NVIDIA H100 Tensor Core GPU Datasheet,” <https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet>, 2023.
- [6] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” in *Proceedings of EMNLP*, 2023.
- [7] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” *arXiv preprint arXiv:1911.02150*, 2019.
- [8] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” *arXiv preprint arXiv:2405.04434*, 2024.
- [9] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-free asymmetric 2bit quantization for KV cache,” in *Proceedings of ICML*, 2024.
- [10] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [11] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *Proceedings of ICLR*, 2024.
- [12] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [13] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “SnapKV: LLM knows what you are looking for before generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of SOSP*, 2023.

- [15] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [16] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proceedings of ICML*, 2023.
- [17] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, “CacheGen: KV cache compression and streaming for fast large language model serving,” in *Proceedings of ACM SIGCOMM*, 2024.
- [18] LMCache Team, “LMCache: An efficient KV cache layer for enterprise-scale LLM inference,” <https://github.com/LMCache/LMCache>, 2024.
- [19] Y. Xu, N. K. Khaira, and T. Singh, “KV cache optimization strategies for scalable and efficient LLM inference,” *arXiv preprint arXiv:2603.20397*, 2026.
- [20] NVIDIA Corporation, “NVIDIA Blackwell B200 Tensor Core GPU Datasheet,” <https://www.nvidia.com/en-us/data-center/b200/>, 2024.
- [21] Advanced Micro Devices, “AMD Instinct MI300X Accelerator Datasheet,” <https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html>, 2024.
- [22] H. Touvron, L. Martin, K. Stone *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [23] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [24] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “QServe: W4A8KV4 quantization and system co-design for efficient LLM serving,” in *Proceedings of MLSys*, 2025.
- [25] M. Adnan, A. Arunkumar, G. Jain, P. Nair, I. Soloveychik, and P. Kamath, “Keyformer: KV cache reduction through key tokens selection for efficient generative inference,” in *Proceedings of MLSys*, 2024.
- [26] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu, and W. Xiao, “PyramidKV: Dynamic KV cache compression based on pyramidal information funneling,” *arXiv preprint arXiv:2406.02069*, 2024.
- [27] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, “Quest: Query-aware sparsity for efficient long-context LLM inference,” in *Proceedings of ICML*, 2024.
- [28] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [29] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, “CacheBlend: Fast large language model serving for RAG with cached knowledge fusion,” in *Proceedings of EuroSys*, 2025.
- [30] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, “Model tells you what to discard: Adaptive KV cache compression for LLMs,” in *Proceedings of ICLR*, 2024.
- [31] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [32] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *Proceedings of OSDI*, 2022.
- [33] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proceedings of ISCA*, 2024.
- [34] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proceedings of OSDI*, 2024.
- [35] G. von Laszewski, W. Brewer, J. Thiyyagalingam, J. Papay, A. Foundjem *et al.*, “AI benchmark democratization and carpentry,” *arXiv preprint arXiv:2512.11588*, 2025.